

HOSPITAL MANAGEMENT SYSTEM

*A Project Report Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted By

ADABALA APARNA LATHA 22221A0501

DEVUDU BALA DHANA GANESH 22221A0526

ANNAMNEEDI VEERA APARNA 22221A0508

Under the Esteemed Guidance of

Dr.MEERASHARIF SHEIK

M.Tech.Ph.D

Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Affiliated to J.N.T.U.K, Kakinada)

(Accredited by NBA & NAAC with “A” Grade)

ODALAREVU-533210

2022 – 2026

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE
(AUTONOMOUS)
ODALAREVU-533210
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled **“HOSPITAL MANAGEMENT SYSTEM”** being submitted for the partial fulfillment of the requirements for the award of the degree of Bachelor Of Technology in **Computer Science and Engineering(CSE)** to Bonam Venkata Chalamayya Engineering College(A), Odalarevu, is a Bonafide work done by **ADABALA APARNA LATHA (22221A0501), DEVUDU BALA DHANA GANESH(22221A0526), ANNAMNEEDI VEERA APARNA (22221A0508)** under my guidance during the academic year **2022-2026** and it has been found suitable for acceptance according to the requirement of the University.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

Project Guide
Dr. MEERASHARIF SHEIK
M.Tech.,Ph.D

Professor & HOD

Head of the Department
Dr. MEERASHARIF SHEIK
M.Tech., Ph.D,

Professor & HOD

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We profoundly grateful to express our deep sense of gratitude and respect towards our guide **Dr. MEERASHARIF SHEIK**, **M.Tech,Ph.D.**, **Professor** of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for her excellent guidance right from selection of the project and her valuable suggestions throughout the project work. Her constant encouragement and support has been the cause of our success, in completing this thesis in the college.

We also grateful to the Project Co-Ordinator **Mr.M.V.K.SUBASH**, **M.Tech., (Ph.D)**, **Assistant Professor** of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work.

We really thankful to **Dr. MEERASHARIF SHEIK**, **M.Tech., Ph.D**, **Professor** and **Head of the Department for Computer Science and Engineering** for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to **Dr. MAHESWAR DUTTA**, **M.E., Ph.D**, **Principal**, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his support during and till the completion of the project.

We are also thankful to our **MANAGEMENT**, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the **TEACHING AND NON-TEACHING STAFF** of Computer Science and Engineering Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, and **FRIENDS** for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our **FAMILY MEMBERS, CLASS MATES AND TEACHERS** from our child hood, whose vision, love and inspiration made us to reach out for these glories.

Project Associates.....

ADABALA APARNA LATHA	22221A0501
DEVUDU BALA DHANA GANESH	22221A0526
ANNAMNEEDI VEERA APARNA	22221A0508

ABSTRACT

In this paper, we present a resource that addresses the growing need for efficient and digital solutions in the healthcare sector. Traditional hospital management systems often rely on manual record-keeping, paper-based patient files, phone-based appointment scheduling, and fragmented platforms for medical services. These methods frequently result in appointment delays, difficulty in tracking patient medical history, inefficiencies in treatment management, and overall inconvenience for patients, doctors, and hospital staff. To overcome these challenges, this project proposes a Smart Hospital Management System, a centralized and intelligent platform designed to modernize and streamline healthcare management.

The Smart Hospital Management System is a comprehensive web-based application that enables patients, doctors, and administrators to manage hospital operations through a single platform. The system allows patients to securely register and create profiles containing essential details such as personal information, medical history, prescriptions, test reports, and treatment records. Automated notifications and reminders help patients keep track of appointments, follow-up visits, and medication schedules, thereby improving healthcare efficiency and continuity of care.

The system also provides an effective appointment scheduling module that supports both in-hospital consultations and teleconsultation services. This feature reduces waiting times, improves accessibility to medical professionals, and simplifies appointment management for both patients and doctors. Additionally, the application supports digital access to medical reports, prescriptions, and billing information, ensuring transparency and ease of use.

The application is developed using Java Full Stack technologies, with Spring Boot and REST APIs forming the backend, MySQL serving as the database, and React powering the frontend interface. Security is implemented using Spring Security and JWT-based authentication, enabling role-based access for patients, doctors, and administrators. Overall, the Smart Hospital Management System provides a scalable, secure, and user-friendly digital solution that enhances efficiency, accuracy, and accessibility in modern healthcare management.

.

TABLE OF CONTENTS PAGE NO

ABSTRACT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	v

CHAPTER – 1

INTRODUCTION	7
--------------------	---

CHAPTER – 2

LITERATURE SURVEY	9
-------------------------	---

CHAPTER – 3

SYSTEM STUDY AND FEASIBILITY STUDY	11
--	----

CHAPTER – 4

SYSTEM REQUIREMENTS SPECIFICATION	12
4.1 Existing Hospital Management System	12
4.2 Proposed Hospital Management System	12
4.3 Hardware & Software Requirements	14

CHAPTER – 5

SYSTEM ARCHITECTURE	15
5.1 UML Diagrams	16
5.1.1 Use Case Diagram (Patient, Doctor, Admin)	17
5.1.2 Class Diagram	18
5.1.3 Sequence Diagram	19
5.1.4 Collaboration Diagram	20
5.1.5 Data Flow Diagram	22

CHAPTER – 6

IMPLEMENTATION	23
6.1 Modules of Hospital Management System	23

- Patient Management Module
 - Doctor Management Module
 - Appointment Scheduling Module
 - Medical Records & Prescription Module
 - Admin Management Module
-

CHAPTER – 7

SOFTWARE ENVIRONMENT	24
7.1 Introduction to Software Environment	24
7.1.1 Operating Systems	24
7.2 Java Programming Language	24
7.2.1 Overview of Java	24
7.2.2 Key Features of Java	25
7.3 Spring Boot Framework	25
7.3.1 Introduction to Spring Boot	25

7.3.2	Role of Spring Boot in the Project	26
7.3.3	Spring MVC Architecture	26
7.3.4	Advantages of Spring Boot	27
7.4	Frontend Technologies	28
7.4.1	React Framework	28
7.4.2	TypeScript and TSX	29
7.4.3	CSS for User Interface Design	30
7.4.4	Advantages of React Framework	30
7.5	Backend Technologies	31
7.5.1	RESTful Web Services	31
7.5.2	Spring Data JPA	32
7.5.3	Hibernate ORM	32
7.6	Database Management System	33
7.6.1	MySQL Database	33
7.6.2	Advantages of MySQL	34

CHAPTER – 8

SYSTEM STRUCTURE AND SAMPLE CODE	35
----------------------------------	----

CHAPTER – 9

SYSTEM TESTING	51
Types of Testing	51
9.1 Unit Testing	51
9.2 Integration Testing	52
9.3 Functional Testing	53
9.4 System Testing	53
9.5 White Box Testing	53
9.6 Black Box Testing	53

9.7	Regression Testing		54
9.8	Acceptance Testing		54
9.9	Test Strategy and Approach		54

CHAPTER – 10

RESULTS AND OUTPUT SCREENS		55
----------------------------	-------	----

CHAPTER – 11

CONCLUSION		63
------------	-------	----

CHAPTER – 12

REFERENCES		65
------------	-------	----

LIST OF FIGURES

FIGURE NO	FIGURE TITLE	PAGE NO
Fig 5.1	System Architecture	15
Fig 5.2	Use Case Diagram (Patient, Doctor, Admin).....	17
Fig 5.3	Class Diagram	18
Fig 5.4	Sequence Diagram.....	19
Fig 5.5	Collaboration Diagram.....	20
Fig 5.6	Data Flow Diagram – Level 0	21
Fig 5.7	Data Flow Diagram – Level 1	21
Fig 5.8	Deployment Diagram	22
Fig 10.1	Login Page.....	55
Fig 10.2	Role Selection for Registartion	56
Fig 10.3	Patient Registration	56
Fig 10.4	Doctor Registration	57
Fig 10.5	Appointment Booking Page(Patient Side)	58
Fig 10.6	Patient Dashboard	59
Fig 10.7	Doctor Dashboard(Doctor Portal)	60
Fig 10.8	Admin Dash board(HMS Portal).....	61
Fig 10.9	Manage Doctors and Patients Page(AdminView).....	62

CHAPTER – 1

INTRODUCTION

Hospital management is a critical component of the healthcare industry, involving the efficient handling of user information, medical staff records, and patient data. With advancements in web technologies, modern Hospital Management Systems have become essential for automating hospital operations and improving service quality. A centralized digital system helps reduce manual workload, minimize errors, and ensure secure and reliable access to hospital data.

This project explores the development of a web-based Hospital Management System that automates key hospital processes such as user registration, authentication, and role-based data management. By implementing separate modules for users, doctors, and patients, the system efficiently manages hospital-related information while ensuring data accuracy and consistency. The digital platform enhances accessibility and reduces dependency on traditional paper-based systems.

The Hospital Management System is developed using Java Full Stack technologies, with Spring Boot for backend services, React (Vite) for the frontend interface, and MySQL for database management. The system follows a RESTful architecture for seamless communication between the frontend and backend, along with CORS configuration to support cross-origin interactions. These technologies enable secure, scalable, and high-performance application development.

This project focuses on designing and implementing a secure and scalable Hospital Management System that supports role-based access control and centralized data management. The objective is to improve operational efficiency, enhance data security, and provide a flexible foundation that can be extended with advanced features such as appointment scheduling, billing, and reporting in the future.

CHAPTER - 2

LITERATURE SURVEY

The advancement of digital technologies has significantly transformed the healthcare sector, particularly in the way patients access medical information and manage hospital services. With the widespread adoption of web-based applications and online healthcare platforms, patients increasingly rely on digital systems for appointment scheduling, medical consultations, accessing electronic health records, and managing billing services. While this digital transformation has improved accessibility, efficiency, and patient engagement, it has also introduced challenges related to data security, information accuracy, interoperability between departments, and seamless service integration. Several studies have examined these challenges and proposed technological solutions to enhance hospital management systems.

A. Research Related to Digital Hospital Management Systems

In 2018, A. Smith conducted a study on the effectiveness of digital hospital management systems in supporting patient appointment scheduling and maintaining electronic medical records. The study highlighted that automated appointment systems and reminder notifications significantly reduced missed appointments and improved overall patient satisfaction. However, it also identified limitations in integrating data across multiple hospital departments.

Subsequently, B. Lee (2019) analysed patient interactions with online hospital portals and healthcare applications. The research revealed that patients often experienced issues due to inconsistent medical records and delayed updates from healthcare providers. The study emphasized the importance of centralized hospital information systems to ensure data accuracy, real-time access, and continuity of patient care.

In 2020, C. Johnson proposed a centralized hospital management framework that integrated patient records, doctor schedules, laboratory results, and billing information into a unified system. The research demonstrated that centralized data storage improved coordination among healthcare professionals and enhanced treatment efficiency. However, the framework required further improvements in user interface design and scalability to support large hospital environments.

B. Studies on Information Reliability, Security, and Data Validation

Ensuring the accuracy and security of medical information has been a major focus of recent healthcare research. D. Patel et al. (2021) examined the risks associated with improper handling of electronic health records (EHRs) in digital hospital systems. The study found that weak authentication mechanisms and inadequate access control could result in data breaches and compromise patient privacy. The researchers stressed the need for strong authentication, encryption techniques, and role-based access control in hospital management systems.

Similarly, E. Turner (2021) explored the impact of unverified online medical information on patient decision-making. The research revealed that patients often relied on online health-related content without professional consultation, which sometimes led to delayed or incorrect treatments. The study recommended integrating doctor-verified information and secure communication features within hospital platforms to improve trust, safety, and treatment outcomes.

C. Healthcare Service Integration and Accessibility

Improving access to healthcare services through digital platforms has been another important research area. F. Brown and G. Carter (2022) developed a location-based healthcare service discovery system that enabled patients to identify nearby hospitals, clinics, and emergency medical services. Their study showed that such systems significantly reduced response times during emergencies and enhanced accessibility to healthcare facilities. However, the system did not support appointment scheduling or sharing of medical records.

In another study, H. Ramirez (2021) introduced a hybrid hospital management system that combined automated appointment scheduling with manual doctor confirmations. The research demonstrated improved operational efficiency in hospitals and reduced patient waiting times. Despite its effectiveness, the system required further enhancements in data security, system scalability, and integration with electronic medical record systems.

CHAPTER - 3

SYSTEM STUDY FEASIBILITY STUDY

The feasibility of the proposed **Hospital Management System** is analysed during this phase to determine whether the system can be successfully developed and implemented within the hospital environment. A general project plan and preliminary cost estimates are considered to ensure that the system demonstrates practicality, efficiency, and value. During system analysis, a detailed feasibility study is conducted to confirm that the proposed solution does not impose unnecessary financial, technical, or operational burdens on the organization.

For an effective feasibility analysis, a clear understanding of the major system requirements is essential. Based on this analysis, three key feasibility considerations are evaluated:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

Economic Feasibility

Economic feasibility examines the financial impact of the proposed system on the hospital or healthcare organization. Since the budget allocated for system development is limited, all project-related expenses must be carefully evaluated and justified. The primary objective of this study is to ensure that the system can be developed within the available financial resources while providing significant operational benefits.

The proposed **Hospital Management System** is economically feasible as most of the technologies used in development, such as Java, Spring Boot, React, and MySQL, are open-source and freely available. This considerably reduces software licensing and infrastructure costs. Only minimal expenses may be incurred for deployment or optional enhancements. The low development and maintenance cost makes the system affordable and cost-effective for hospital implementation.

Technical Feasibility

Technical feasibility evaluates whether the proposed system can be developed using the available hardware, software, and technical expertise. The Hospital Management System does not require highly specialized or expensive technical infrastructure. It is designed to function efficiently

using commonly available development tools, databases, and server environments.

The system has moderate technical requirements and does not demand major modifications to the existing technical setup of the hospital. It can be deployed on standard servers and accessed through commonly used devices such as computers and mobile systems. Therefore, the technical feasibility of the Hospital Management System is well within acceptable and manageable limits.

Social Feasibility

Social feasibility assesses the level of acceptance of the proposed system among users, including patients, doctors, and administrative staff. It focuses on user comfort, adaptability, and confidence while interacting with the system. The success of the Hospital Management System largely depends on its usability and the willingness of users to adopt digital healthcare solutions. The system is designed with a user-friendly interface that requires minimal training for effective use. Users are not expected to feel threatened by the system; instead, it is introduced as a supportive tool that simplifies hospital operations and improves healthcare service delivery. Proper guidance, awareness, and feedback mechanisms help build user confidence and encourage system adoption. User feedback and suggestions are considered valuable for continuous system improvement.

Conclusion of Feasibility Study

Based on the feasibility analysis, the proposed **Hospital Management System** is found to be economically affordable, technically practical, and socially acceptable. The system satisfies all feasibility requirements and can be successfully implemented without imposing significant constraints on financial resources, technical infrastructure, or users.

CHAPTER - 4

SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 EXISTING SYSTEM:

In the existing scenario, hospitals and clinics largely depend on manual processes, paper-based records, and multiple disconnected software systems to manage healthcare activities. Patient information related to medical history, prescriptions, laboratory reports, and appointments is often recorded in physical files or stored across different systems. While some hospitals use basic digital tools, these systems are often isolated, outdated, and lack proper integration. Most existing healthcare systems do not provide a centralized platform for managing patient records and hospital operations. Doctors and administrative staff rely on physical documents or separate applications to track appointments, treatments, and billing details, which increases the risk of data loss, duplication, and human error. Moreover, current systems typically focus on limited functionalities such as registration or billing and fail to offer a comprehensive solution that integrates patient management, appointment scheduling, medical records, and secure access within a single platform.

DISADVANTAGES:

- 1) The existing system does not provide the capability to efficiently manage, store, and process large volumes of patient data, which affects performance and scalability.
- 2) Presence of unverified, inconsistent, or outdated medical information due to manual record-keeping and lack of centralized validation mechanisms.

4.2 PROPOSED SYSTEM:

The proposed **Hospital Management System** aims to overcome the limitations of the existing system by providing a centralized, secure, and reliable digital platform for hospital and healthcare management. The system enables patients to create detailed profiles, store electronic medical records, prescriptions, laboratory reports, and access treatment history in a structured manner. Automated notifications and reminders help patients manage appointments, follow-up visits, and medication schedules effectively.

The system emphasizes the availability of accurate and verified medical information to prevent misinformation. An administrator-controlled management module ensures that only authorized healthcare professionals can update medical records and treatment details, maintaining data accuracy and reliability.

Additionally, the system supports doctor discovery, appointment scheduling, and notification services to enhance patient convenience and streamline hospital operations. Both in-person consultations and teleconsultation options are supported to improve accessibility to healthcare services.

The proposed system is designed to be scalable, secure, and user-friendly, ensuring ease of adoption by patients, doctors, and administrative staff with minimal technical knowledge. By integrating multiple hospital operations into a single platform, the system improves efficiency, reduces manual effort, and enhances overall healthcare service delivery.

ADVANTAGES

- (i) Centralized patient profile and electronic medical record management.
- (ii) Accurate and verified medical information managed by authorized professionals.
- (iii) Automated reminders for appointments, medications, and follow-up visits.
- (iv) Secure authentication with role-based access control for patients, doctors, and administrators.
- (v) Scalable and maintainable system architecture.
- (vi) Improved patient experience and hospital operational efficiency.

The proposed **Hospital Management System** provides a centralized and secure platform for managing patient records, appointments, prescriptions, and healthcare services. It ensures reliable and verified medical information, reducing dependence on manual and untrusted sources. Automated notifications improve consistency in treatment schedules and follow-ups. The system supports multiple users with role-based access and ensures data security. Overall, it enhances efficiency, accuracy, and quality of healthcare service delivery.

4.3 HARDWARE & SOFTWARE REQUIREMENTS:

HARDWARE REQUIREMENTS

- Processor : Pentium – IV or higher
 - RAM : 4 GB (Minimum)
 - Hard Disk : 20 GB or above
 - Keyboard : Standard Windows Keyboard
 - Mouse : Two or Three Button Mouse
 - Monitor : SVGA Monitor
-

SOFTWARE REQUIREMENTS

- Operating System : Windows 7 / Windows 10 or above
- Programming Language : Java
- Backend Framework : Spring Boot
- Frontend Technology : React using JSX and CSS
- Backend : Spring Boot
- Designing Tools : JSX, CSS
- Database : MySQL

CHAPTER - 5

SYSTEM ARCHITECTURE

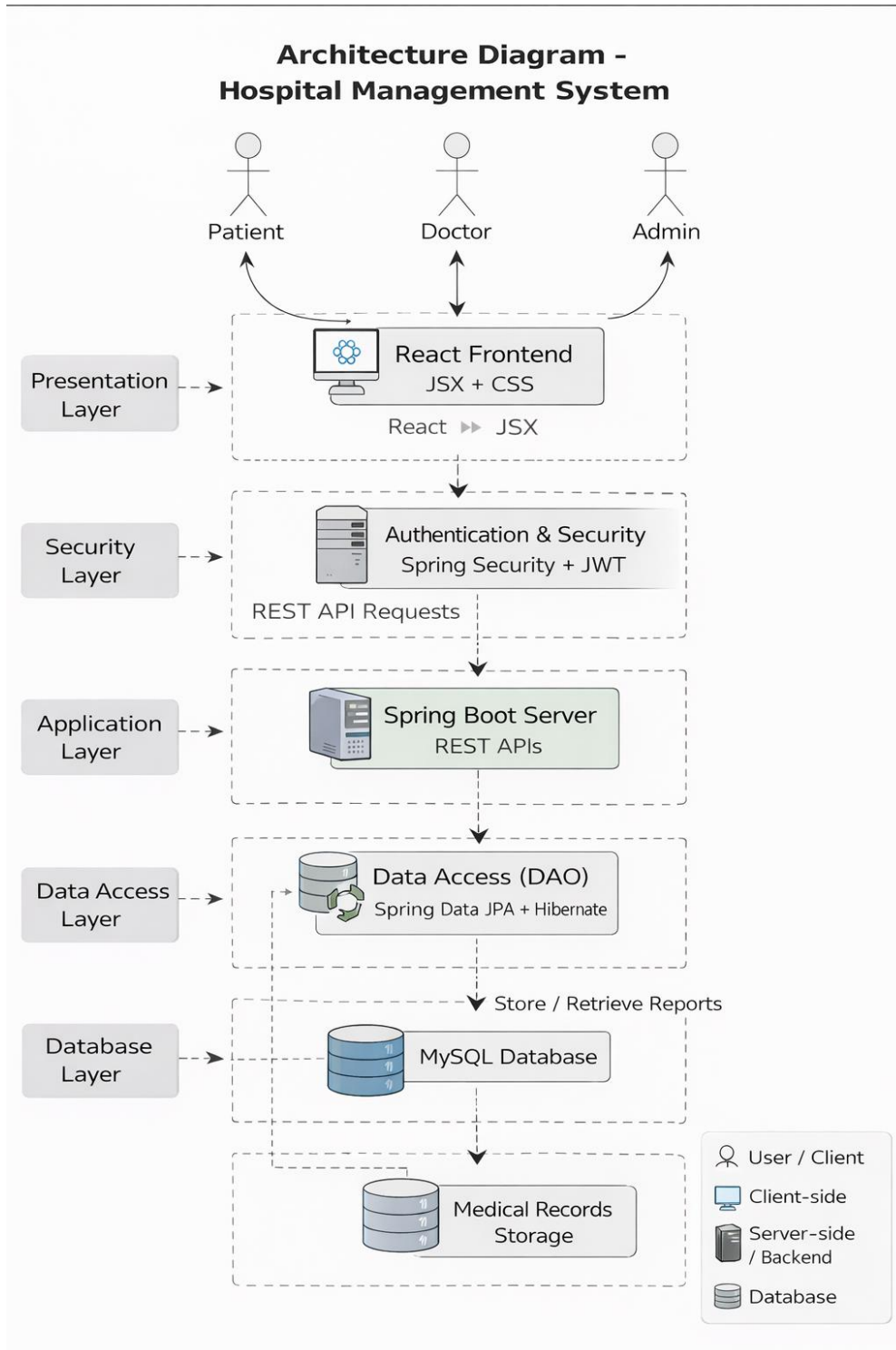


Fig 5.1 System Architecture

5.1 UML DIAGRAM'S:

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.

5.1.1 USE CASE DIAGRAM:

A Use Case Diagram is a behavioral diagram in UML that represents the functional requirements of a system from the user's perspective. It shows how different **actors** interact with the system through various **use cases**. The purpose of a use case diagram is to provide a high-level overview of system functionality and user interactions.

In the **Hospital Management System**, use case diagrams illustrate interactions between actors such as **Patient**, **Doctor**, and **Admin** with system functionalities like registration, appointment

scheduling, viewing medical records, and system management. This diagram helps in understanding system scope and user responsibilities.

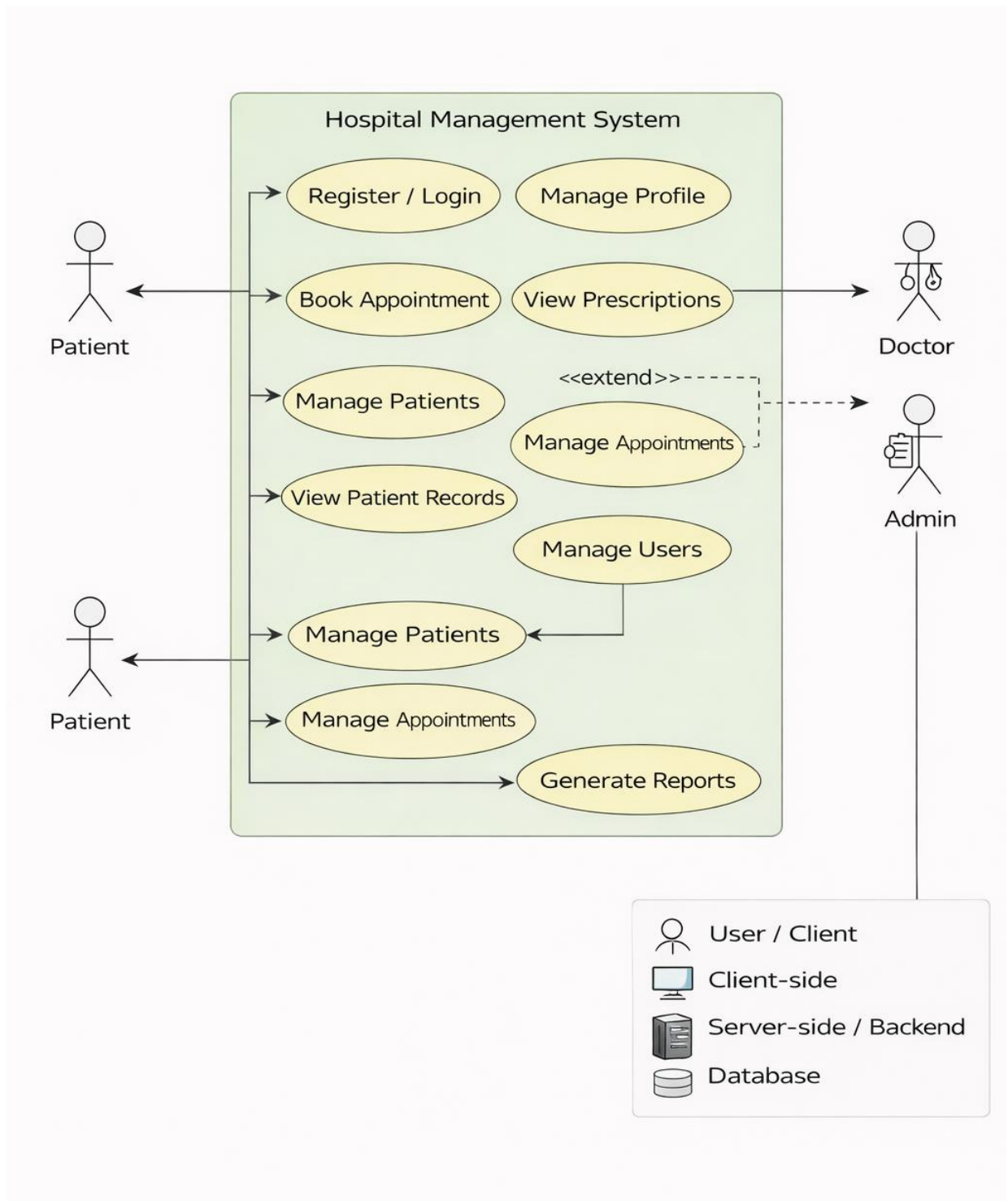


Fig 5.2 Use Case Diagram

5.1.2 CLASS DIAGRAM:

A Class Diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships among classes. It describes how data is organized and how different entities are related within the system.

In the Hospital Management System, class diagrams represent classes such as User, Patient, Doctor, Appointment, and MedicalRecord, along with their attributes and operations. This diagram provides a blueprint of the system structure and helps in database and backend design.

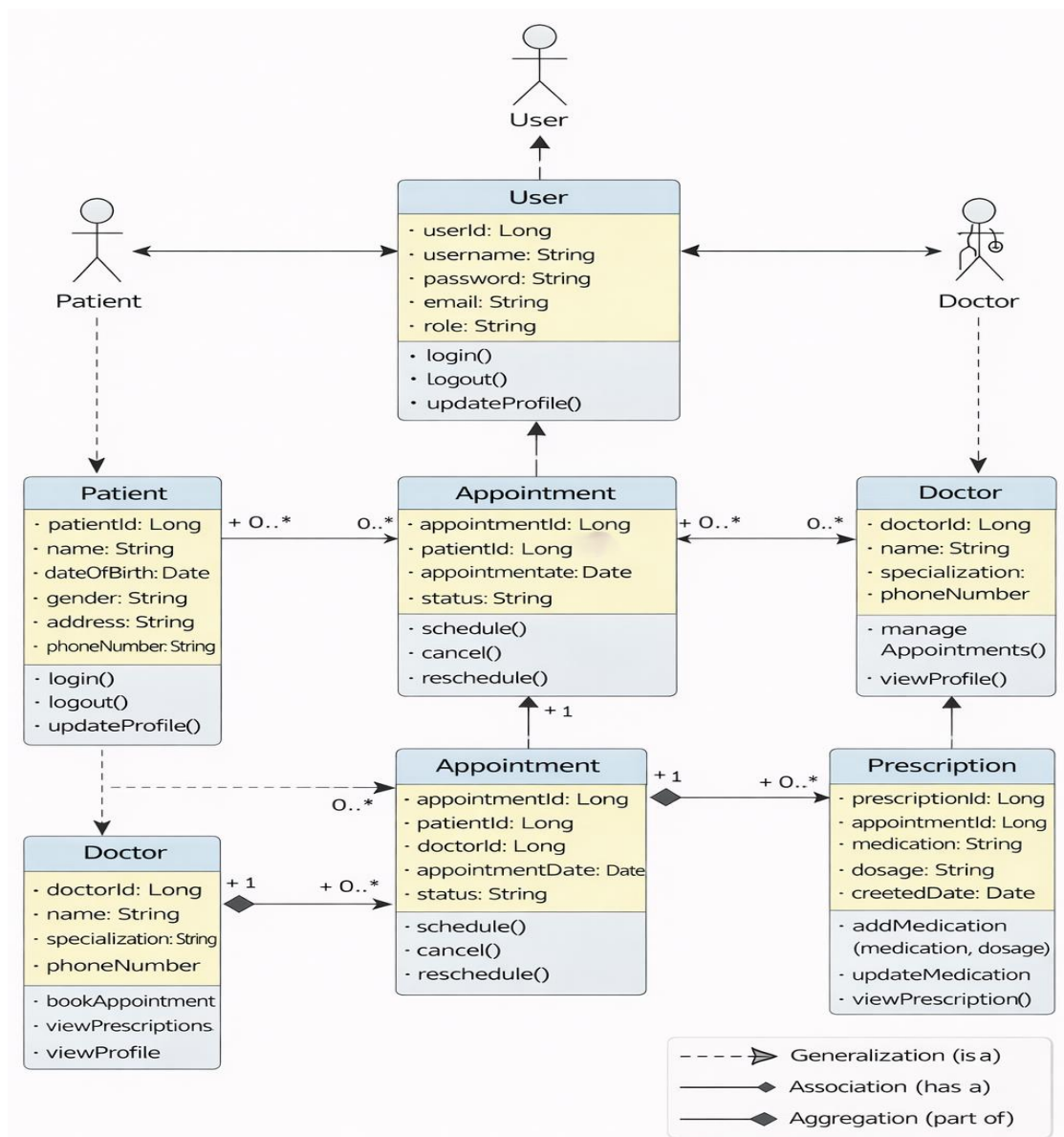


Fig 5.3 Class Diagram

5.1.3 SEQUENCE DIAGRAM:

A Sequence Diagram is an interaction diagram that shows how objects communicate with each other in a specific sequence over time. It focuses on the order of messages exchanged between objects to accomplish a particular functionality.

In the Hospital Management System, sequence diagrams illustrate workflows such as **patient login**, **appointment booking**, and **viewing medical records**, showing interactions between the frontend, backend services, and database. This diagram helps in understanding the dynamic behavior of the system.

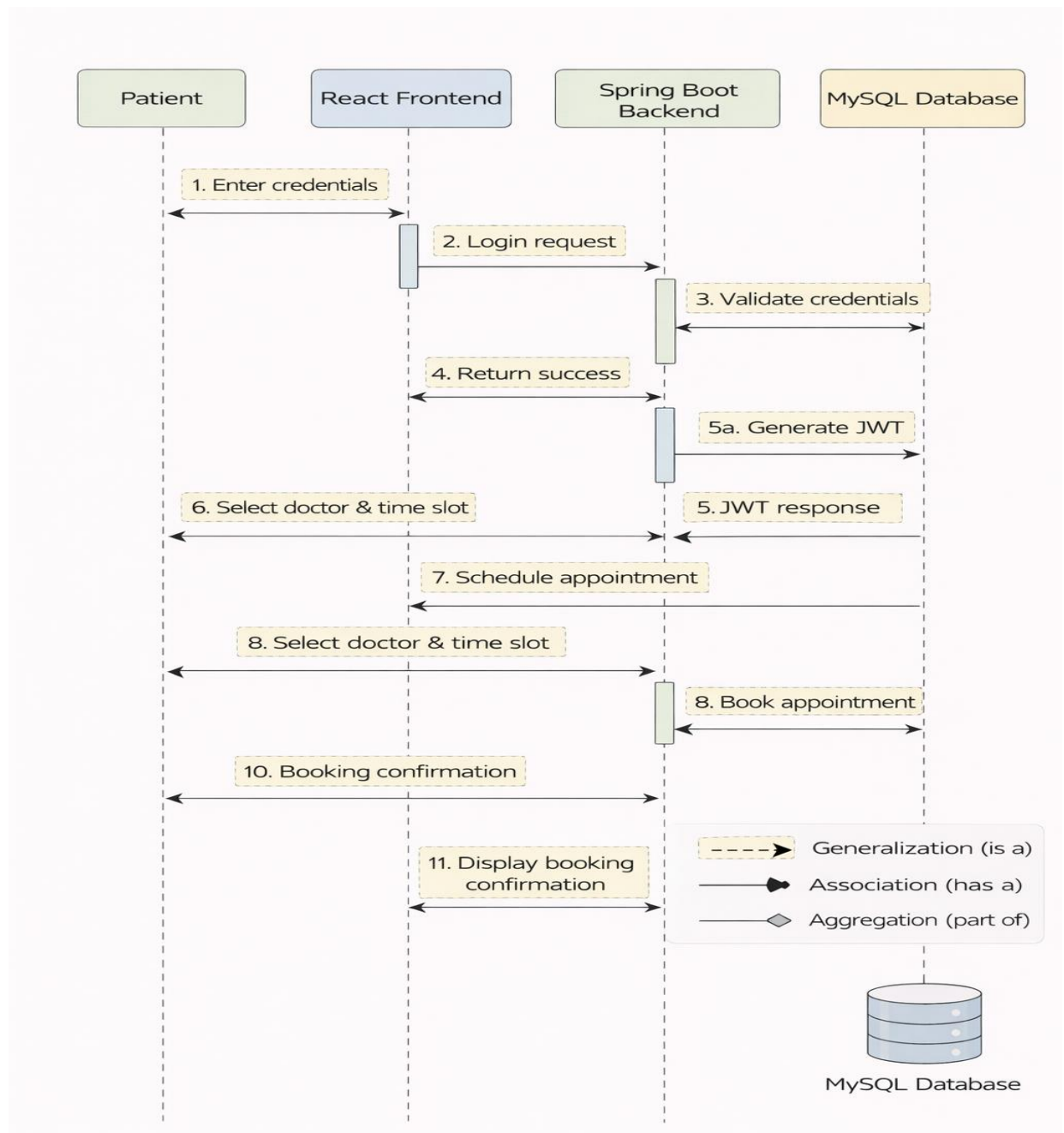


Fig 5.4 Sequence Diagram

COLLABORATION DIAGRAM:

A Collaboration Diagram (also known as a Communication Diagram) shows how objects interact with each other through message exchanges, focusing on relationships rather than sequence. It emphasizes object roles and connections within a system.

In the Hospital Management System, collaboration diagrams depict how components such as user interface, authentication module, application server, and database collaborate to complete operations like secure data access and record management.

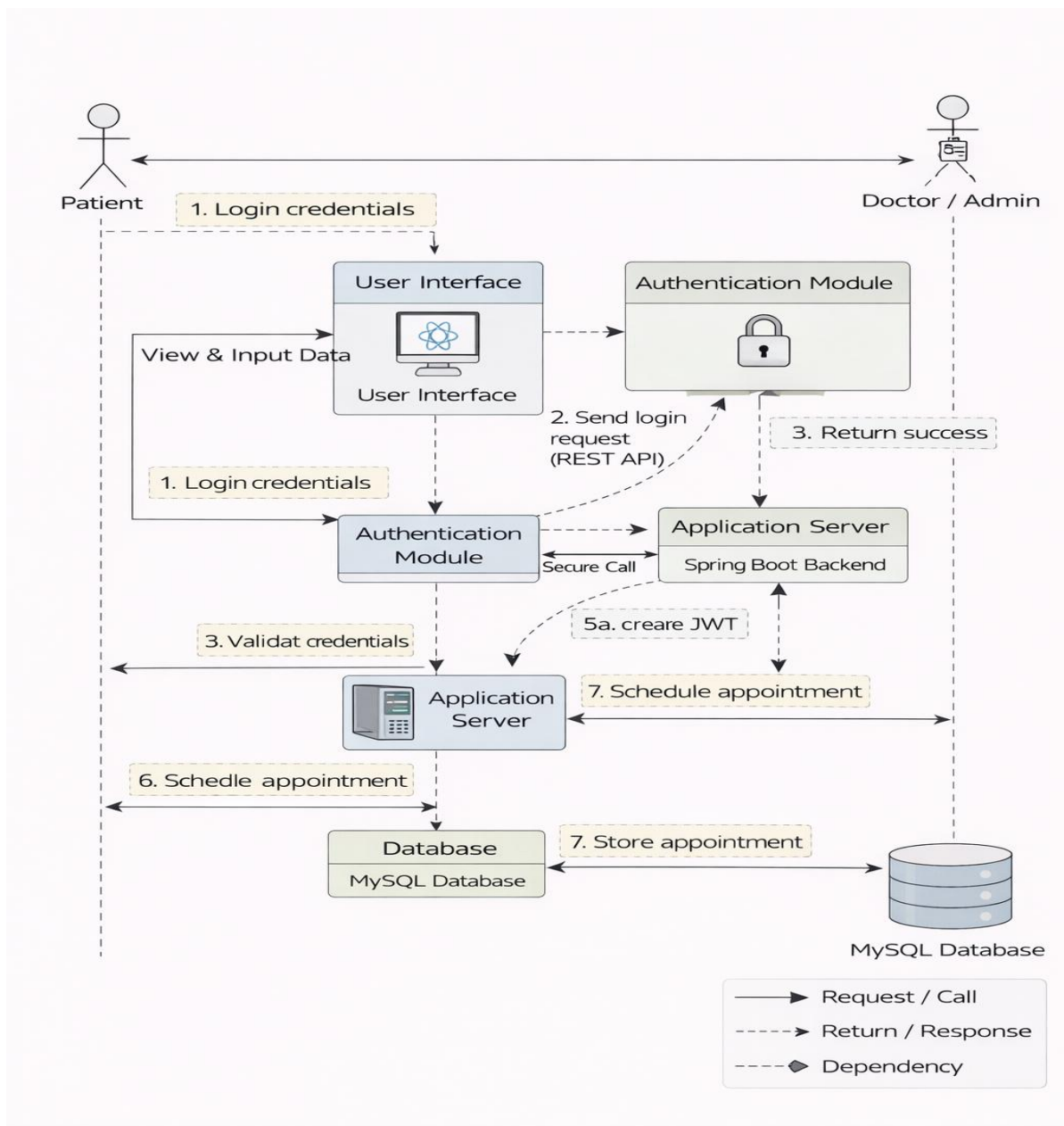


Fig 5.5 Collaboration Diagram

DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) represents how data flows through a system, illustrating the movement of data between external entities, processes, and data stores. It helps in understanding how information is input, processed, stored, and output.

In the Hospital Management System, DFDs represent data flow for processes such as patient registration, appointment scheduling, and medical record management. DFD Level 0 provides a high-level view, while Level 1 offers a more detailed breakdown of processes.

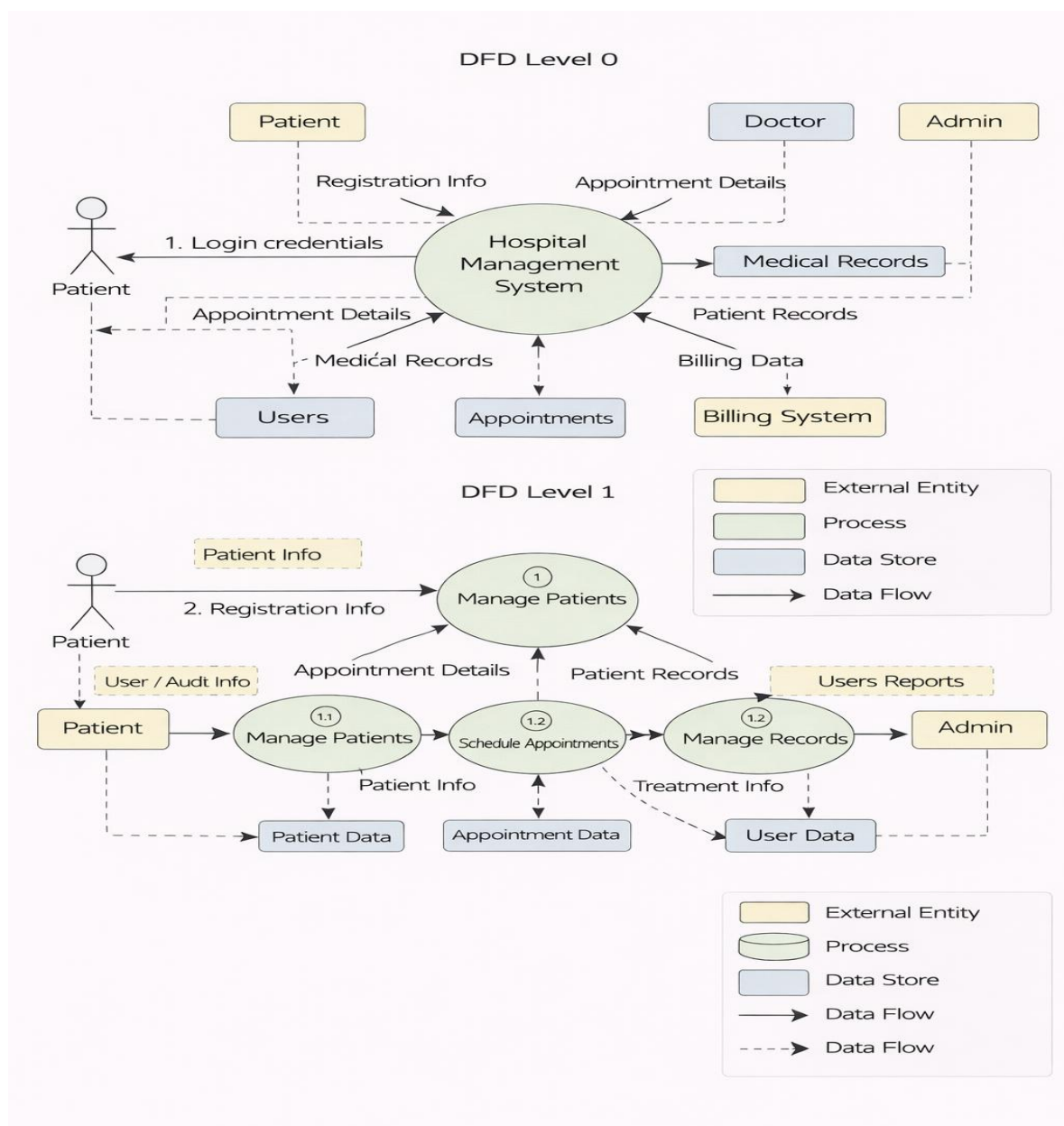


Fig 5.6 Data Flow Diagram

Fig 5.7 Data Flow Diagram

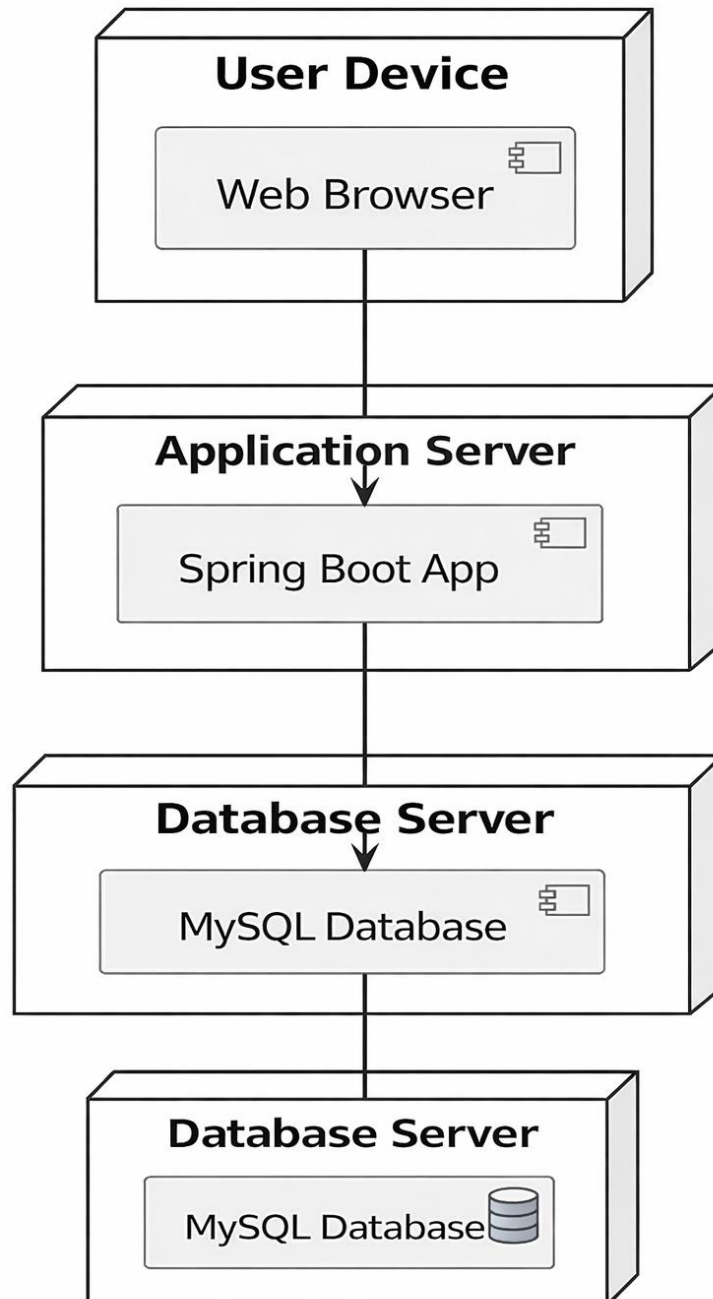


Fig 5.8 Deployment Diagram

CHAPTER – 6

IMPLEMENTATION

6.1 MODULES:

The **Hospital Management System** is designed using a modular approach to ensure clarity, security, and ease of use. The system is divided into three major modules based on user roles: **Patient**, **Doctor**, and **Admin**. Each module performs specific functions to support efficient hospital and healthcare management.

Patient Module

The Patient module is designed for users who seek medical services from the hospital. Patients must register and log in using valid credentials before accessing the system. After successful authentication, patients can create and manage their personal profiles, view and store medical records, prescriptions, and test reports. This module allows patients to book doctor appointments, view appointment status, access treatment history, and manage their profile information. The Patient module plays a key role in ensuring organized access to healthcare services and improving patient experience.

Doctor Module

The Doctor module enables registered medical professionals to securely access the system. After logging in, doctors can view appointment requests submitted by patients and access patient medical records for consultation and treatment planning. Doctors can update appointment status, add prescriptions, and record treatment details. This module improves coordination between patients and doctors while reducing manual record handling and improving treatment accuracy.

Admin Module

The Admin module is responsible for managing and monitoring the overall system. The administrator logs in using authorized credentials and can view all registered users, including patients and doctors. The admin has the authority to approve, block, or manage user accounts, monitor appointments, and oversee system data. This module ensures data integrity, security, and smooth operation of the Hospital Management System.

CHAPTER - 7

SOFTWARE ENVIRONMENT

Introduction to Software Environment:

The software environment plays a critical role in the successful development and execution of any modern application. It defines the collection of programming languages, frameworks, libraries, tools, databases, and operating systems used throughout the software development lifecycle. A well-chosen software environment improves system reliability, performance, scalability, and maintainability.

The **Hospital Management System** is designed as a full-stack web application that supports **patients, doctors, and administrators**. The application manages patient profiles, medical records, appointments, and administrative operations through a secure and scalable architecture. To achieve these objectives, the system uses **Java 17, Spring Boot, React with JSX, and MySQL**, along with supporting tools and frameworks.

This chapter provides a detailed explanation of the software environment used in the project, including the rationale behind technology selection, architectural benefits, and how each component contributes to the overall system.

7.1.1 Operating System:

Windows Operating System:

The **Hospital Management System** is developed and tested on **Windows 7 Ultimate / Windows 10 and above**. The Windows operating system provides a stable and reliable development environment with extensive support for Java development kits, web servers, databases, and modern development tools. Its compatibility with various IDEs and database management systems makes it suitable for building and testing enterprise-level web applications.

7.2 Java Programming Language:

7.2.1 Overview of Java

Java is a high-level, object-oriented, and platform-independent programming language developed by Sun Microsystems (now Oracle). The Hospital Management System uses **Java 17**, a long-term support (LTS) version that offers enhanced performance, improved security, and long-term stability.

Java follows the principle of **Write Once, Run Anywhere (WORA)**, which allows applications to run on any system that supports the Java Virtual Machine (JVM). This feature makes Java highly suitable for scalable and cross-platform applications such as hospital management systems.

7.2.2 Key Features of Java

- Object-Oriented Programming (OOP)
- Platform independence
- Automatic memory management (Garbage Collection)
- Strong exception handling
- Rich standard libraries
- Multithreading support
- High security

7.3 Spring Boot Framework

7.3.1 Introduction to Spring Boot

Spring Boot is a Java-based framework built on top of the Spring Framework. It simplifies application development by providing features such as auto-configuration, embedded servers, and production-ready tools. Spring Boot eliminates complex XML configurations and allows developers to focus more on implementing business logic.

Spring Boot significantly reduces the complexity involved in developing enterprise-level applications. By providing sensible default configurations, it minimizes manual setup and accelerates the development process. Instead of traditional XML-based configurations, Spring Boot uses annotations and Java-based configuration, resulting in cleaner, more readable, and easier-to-maintain code.

Spring Boot includes an embedded web server such as Apache Tomcat, which enables applications to run independently without requiring external server configuration. This feature simplifies deployment and improves portability across different environments. Additionally, Spring Boot integrates seamlessly with Spring modules such as Spring MVC, Spring Data JPA, and Spring Security, making it well-suited for building secure and scalable applications like the **Hospital Management System**.

Furthermore, Spring Boot provides production-ready features such as health checks, metrics, logging, and application monitoring. These features help developers monitor application performance and ensure system reliability. Overall, Spring Boot enhances developer productivity, reduces development time, and provides a robust foundation for modern, scalable, and secure applications.

7.3.2 Role of Spring Boot in the Project:

Spring Boot plays a central role in the development of the Hospital Management System by acting as the core backend framework. It is responsible for handling all server-side operations, including request processing, business logic execution, data persistence, and security management. The framework enables the development of RESTful web services that allow smooth communication between the React-based frontend and the backend system. In this project, Spring Boot manages user authentication and authorization using **Spring Security and JWT**. It ensures that only authenticated users can access protected resources based on their roles, such as **Patient, Doctor, or Admin**. This role-based access control improves system security and prevents unauthorized access to sensitive medical data. Spring Boot also facilitates efficient database interaction through **Spring Data JPA and Hibernate**. These technologies simplify CRUD (Create, Read, Update, Delete) operations by mapping Java objects to relational database tables, reducing the need for complex SQL queries and maintaining data consistency throughout the application. Another important role of Spring Boot in this project is handling business logic related to patient profile management, medical record storage, appointment scheduling, and hospital administration. The framework supports modular development, allowing different services to be developed, tested, and maintained independently. This modularity enhances maintainability and future scalability. Additionally, Spring Boot's embedded server capability enables the application to run as a standalone service without requiring an external web server. This simplifies deployment and makes the application portable across development, testing, and production environments. Overall, Spring Boot ensures that the **Hospital Management System** is secure, scalable, efficient, and easy to maintain.

7.3.3 Spring MVC (Model–View–Controller):

Spring MVC is a web framework that follows the Model–View–Controller (MVC) architectural pattern, which helps in separating application concerns and improving maintainability. In the Hospital Management System, Spring MVC is used to organize the backend logic in a structured and systematic manner, ensuring a clear separation between data handling, user interface interaction, and request processing.

The Model represents the application data and business logic. In this project, the model consists of entities such as User, Patient, Doctor, MedicalRecord, Appointment, and Prescription. These entities are mapped to relational database tables using Hibernate and Spring Data JPA. The model layer ensures data consistency and encapsulates all business rules related to hospital operations and healthcare management.

The View represents the user interface of the application. In the Hospital Management System, the view is implemented using React with JSX and CSS, which interacts with the backend through RESTful APIs. Although Spring MVC traditionally supports server-side views, in this project it efficiently supports a client-side frontend by acting as a communication bridge between the React interface and backend services.

The Controller acts as an intermediary between the model and the view. Controllers in Spring MVC handle incoming HTTP requests from the frontend, process user input, invoke appropriate business logic from the service layer, and return responses in JSON format. Each controller is responsible for managing specific functionalities such as user authentication, patient management, appointment scheduling, and medical record handling.

By using Spring MVC, the application achieves better modularity, easier debugging, and improved scalability. The MVC pattern also simplifies future enhancements, as changes in one layer do not directly affect other layers. Overall, Spring MVC provides a clean and organized architecture that supports efficient development and long-term maintenance of the Hospital Management System.

7.3.4 Advantages of Spring Boot:

Spring Boot offers several advantages that make it an ideal framework for developing the Smart Pet Care Application. One of the primary benefits of Spring Boot is its ability to significantly reduce development time through auto-configuration and opinionated defaults. By automatically configuring commonly used components, Spring Boot minimizes manual setup and allows developers to concentrate on implementing core application functionality.

Another major advantage of Spring Boot is its support for embedded servers such as Apache Tomcat. This eliminates the need for external server configuration and allows the application to run as a standalone service. As a result, deployment becomes simpler and more

consistent across different environments, including development, testing, and production.

Spring Boot provides seamless integration with other Spring modules such as Spring MVC, Spring Data JPA, and Spring Security. This integration enables efficient handling of web requests, database operations, and security mechanisms within a unified framework. In the Smart Pet Care Application, this allows smooth interaction between frontend services, backend logic, and persistent data storage.

Additionally, Spring Boot supports microservice-friendly architecture, which improves scalability and maintainability. The modular structure allows individual components such as user management, pet profile services, and notification services to be developed and maintained independently. Built-in production-ready features such as health checks, metrics, and logging further enhance system monitoring and reliability.

Overall, Spring Boot improves developer productivity, ensures application security, simplifies deployment, and provides a robust foundation for building scalable and maintainable enterprise-level applications like the Smart Pet Care Application.

7.4 Frontend Technologies:

7.4.1 React Framework:

React is a popular open-source JavaScript library developed by Facebook for building interactive and dynamic user interfaces. In the Hospital Management System, React is used as the primary frontend framework to create a responsive, user-friendly, and efficient user interface. React follows a component-based architecture, which allows the application interface to be divided into small, reusable components such as login forms, patient profile pages, appointment booking screens, dashboards, and administrative panels.

One of the key features of React is the **Virtual DOM**, which enhances application performance by minimizing direct updates to the actual DOM. Instead of reloading entire pages, React updates only the components that change, resulting in faster rendering and a smoother user experience. This is especially beneficial for the Hospital Management System, where users frequently interact with forms, lists, and real-time data displays.

React also supports **unidirectional data flow**, which makes application behaviour more predictable and easier to debug. State management in React allows the application to efficiently handle user inputs, session data, and dynamic content updates. When combined with **JSX**, React improves code readability, component reusability, and overall maintainability of the frontend codebase.

Furthermore, React integrates seamlessly with **RESTful APIs** developed using Spring Boot.

The frontend communicates with the backend through HTTP requests to fetch and update data related to patient profiles, medical records, appointments, and administrative operations. Overall, the React framework provides a scalable, maintainable, and high-performance frontend solution that enhances usability and supports the functional requirements of the **Hospital Management System**.

7.4.2 JSX (JavaScript XML)

JSX (JavaScript XML) is a syntax extension for JavaScript commonly used with the React framework. It allows developers to write HTML-like code directly within JavaScript files, making the user interface code more readable and easier to understand. In the **Hospital Management System**, JSX is used to build dynamic and interactive frontend components. By using JSX, the frontend logic and user interface structure are combined in a single file, which improves code organization and component reusability. Components such as login forms, patient dashboards, appointment booking pages, and admin panels are developed using JSX, making the application modular and maintainable.

One of the major advantages of JSX is improved readability and simplicity. Developers can clearly visualize the UI structure while writing JavaScript logic, which reduces development complexity and errors. JSX also supports conditional rendering and dynamic content updates, which are essential for handling real-time data such as appointments, medical records, and user interactions in the Hospital Management System.

JSX integrates seamlessly with React's component-based architecture and works efficiently with RESTful APIs provided by the Spring Boot backend. Overall, the use of JSX enhances frontend development efficiency, improves user experience, and ensures a scalable and maintainable user interface for the **Hospital Management System**.

Benefits of JSX:

JSX combines the simplicity of HTML-like syntax with the power of JavaScript, making it an effective choice for developing React-based applications. One of the primary benefits of JSX is improved code readability, as developers can clearly visualize the user interface structure while writing JavaScript logic. This reduces complexity and makes the frontend code easier to understand.

Another major benefit of JSX is better code organization and maintainability. By allowing UI logic and structure to exist within the same component, JSX promotes a component-based architecture. This is especially useful in applications like the **Hospital Management System**, where multiple components such as patient dashboards, appointment pages, and admin panels

interact with backend services.

JSX also improves developer productivity by simplifying UI development. It supports conditional rendering and dynamic content updates, enabling efficient handling of real-time data such as appointments, medical records, and user interactions. JSX integrates seamlessly with modern development tools and React libraries, making debugging and UI updates easier. Additionally, JSX ensures scalability by supporting reusable components, which allows the application to grow with new features while maintaining a clean and consistent code structure. Overall, JSX provides a simple, efficient, and maintainable approach to frontend development, contributing to a responsive user interface and improved user experience in the **Hospital Management System**.

7.4.3 CSS for UI Design:

Cascading Style Sheets (CSS) is a styling language used to define the visual presentation of web applications. In the Smart Pet Care Application, CSS is used to design an attractive, consistent, and responsive user interface. It controls the layout, colours, fonts, spacing, and overall look and feel of the application, ensuring a pleasant user experience across different devices and screen sizes.

CSS plays a crucial role in making the application responsive by adapting the layout to various screen resolutions such as desktops, tablets, and mobile devices. Through the use of flexible layouts, media queries, and responsive design techniques, the Smart Pet Care Application provides a seamless and user-friendly interface for all users. This responsiveness is particularly important for pet owners who may access the application from different devices.

In addition, CSS helps maintain a clear visual hierarchy within the application. Elements such as navigation menus, forms, buttons, and alerts are styled consistently to improve usability and accessibility. Proper use of colours, fonts, and spacing ensures that important information such as reminders, notifications, and medical records is easily readable and visually distinct.

By separating presentation from logic, CSS enhances maintainability and reusability of the user interface. Changes to the design can be made without affecting application functionality. Overall, CSS contributes significantly to the usability, accessibility, and professional appearance of the Smart Pet Care Application.

7.4.4 Advantages of React Frontend:

React offers several advantages that make it an effective frontend framework for the Smart Pet Care Application. One of the major benefits of React is its **component-based**

architecture, which allows the user interface to be divided into reusable and independent components. This modular approach simplifies development, improves code reusability, and makes the application easier to maintain and extend.

Another important advantage of React is its **high performance**, achieved through the use of the Virtual DOM. Instead of updating the entire webpage, React updates only the components that change, resulting in faster rendering and smoother user interactions. This is particularly beneficial for the Smart Pet Care Application, where users frequently interact with dynamic elements such as pet profiles, appointment lists, and reminders.

React also provides **better state management** and predictable data flow, which makes application behaviour easier to understand and debug. Combined with TypeScript and TSX, React ensures improved code quality and reduced runtime errors. Additionally, React integrates seamlessly with RESTful APIs developed using Spring Boot, enabling efficient communication between the frontend and backend.

Furthermore, React supports responsive design and enhances user experience across different devices and screen sizes. Its strong community support, extensive libraries, and continuous updates make React a reliable and future-ready frontend solution. Overall, React improves usability, performance, scalability, and maintainability of the Smart Pet Care Application.

7.5 Backend Technologies:

7.5.1 RESTful Web Services:

RESTful Web Services are an architectural style used to build scalable and efficient web applications by enabling communication between client and server over standard HTTP protocols. REST stands for **Representational State Transfer**, and it follows a stateless client–server communication model. In the Smart Pet Care Application, RESTful web services are used to enable seamless interaction between the React-based frontend and the Spring Boot backend.

Each functionality of the application, such as user authentication, pet profile management, appointment scheduling, and reminder notifications, is exposed as a REST API endpoint. These APIs use standard HTTP methods such as **GET, POST, PUT, and DELETE** to perform operations on application resources. Data is exchanged in **JSON format**, which is lightweight, easy to parse, and widely supported across platforms.

One of the key advantages of RESTful web services is their **stateless nature**, where each request contains all the necessary information for processing. This improves scalability and

allows the server to handle multiple concurrent users efficiently. REST APIs also simplify frontend-backend integration, enabling the Smart Pet Care Application to support future extensions such as mobile applications or third-party service integration.

Additionally, RESTful services improve maintainability by separating concerns between the user interface and business logic. They also enhance security when combined with authentication mechanisms like JWT. Overall, RESTful web services provide a flexible, scalable, and reliable communication layer for the Smart Pet Care Application.

7.5.2 Spring Data JPA:

Spring Data JPA is a powerful persistence framework that simplifies database interactions in Java applications by providing an abstraction over the Java Persistence API (JPA). In the Smart Pet Care Application, Spring Data JPA is used to manage data access and persistence operations between the application and the MySQL database. It significantly reduces the amount of boilerplate code required to perform common database operations.

Spring Data JPA allows developers to define repository interfaces that automatically provide built-in methods for Create, Read, Update, and Delete (CRUD) operations. This eliminates the need to write repetitive SQL queries or JDBC code. By using method naming conventions, Spring Data JPA can generate queries dynamically, making database interaction more efficient and readable.

In this project, Spring Data JPA works closely with Hibernate to map Java entity classes such as User, Pet, Medical Record, Appointment, and Reminder to corresponding database tables. It ensures data consistency and simplifies transaction management. Spring Data JPA also supports pagination, sorting, and custom queries, which are useful for handling large datasets and improving application performance.

Overall, Spring Data JPA enhances developer productivity, improves code maintainability, and ensures reliable data persistence, making it a suitable choice for implementing the data access layer of the Smart Pet Care Application.

7.5.3 Hibernate ORM:

Hibernate ORM (Object-Relational Mapping) is a widely used framework that simplifies the interaction between Java applications and relational databases. In the Smart Pet Care Application, Hibernate ORM is used as the underlying persistence framework to map Java objects to database tables and manage database operations efficiently. It acts as a bridge

between the object-oriented domain model of the application and the relational structure of the MySQL database.

Hibernate eliminates the need for writing complex and repetitive SQL queries by automatically handling data persistence using object-oriented principles. Java entity classes such as User, Pet, Medical Record, Appointment, and Reminder are mapped to corresponding database tables through annotations. This approach improves code readability and ensures better maintainability by keeping database logic separate from business logic.

Another important feature of Hibernate is its support for automatic table generation and schema management. This allows developers to easily create or update database tables based on entity definitions. Hibernate also provides powerful caching mechanisms that improve application performance by reducing the number of database hits. Additionally, it manages database transactions effectively, ensuring data consistency and integrity.

Hibernate's integration with Spring Data JPA further enhances development efficiency by providing seamless data access and simplified configuration. Overall, Hibernate ORM plays a vital role in managing data persistence, improving performance, and ensuring scalability in the Smart Pet Care Application.

7.6 Database Management System:

7.6.1 MySQL Database:

MySQL is a widely used open-source relational database management system known for its reliability, performance, and scalability. In the Smart Pet Care Application, MySQL serves as the primary database for storing and managing all critical application data. This includes user information, pet profiles, medical and vaccination records, appointment schedules, reminders, and notification details.

MySQL supports structured data storage using tables, relationships, and constraints, which ensures data consistency and integrity. The relational nature of MySQL allows efficient handling of complex queries and relationships between different entities such as users, pets, and appointments. By using primary keys, foreign keys, and indexing, MySQL enables fast data retrieval and optimized query performance.

In this project, MySQL integrates seamlessly with Spring Data JPA and Hibernate ORM, allowing object-oriented Java entities to be mapped directly to database tables. This integration simplifies data access and reduces manual database management efforts. MySQL also provides robust transaction management, ensuring that critical operations such as appointment booking

and medical record updates are executed safely and reliably.

Overall, MySQL provides a secure, efficient, and scalable data storage solution, making it well-suited for managing the structured and sensitive data required by the Smart Pet Care Application.

7.6.2 Advantages of MySQL:

- High performance
- Data consistency
- Secure transactions
- Easy backup and recovery

H2 is a lightweight, open-source, in-memory relational database that is commonly used for development and testing purposes. In the Smart Pet Care Application, the H2 database is used during the testing phase to validate application functionality without affecting the production database. Its in-memory nature allows data to be stored temporarily during application runtime, enabling faster execution and simplified testing.

One of the main advantages of H2 is its ease of configuration and minimal resource requirements. It does not require a separate server installation and can be embedded directly within the application. This makes it ideal for rapid development, unit testing, and integration testing. Developers can quickly test features such as user authentication, pet profile management, and appointment scheduling in an isolated environment.

H2 is fully compatible with SQL standards and integrates seamlessly with Spring Boot, Spring Data JPA, and Hibernate ORM. This compatibility ensures that database operations tested using H2 behave similarly to those in MySQL, reducing the risk of database-related issues during deployment. Overall, the H2 database improves development efficiency, supports faster testing cycles, and enhances the reliability of the Smart Pet Care Application.

CHAPTER – 8

STRUCTURE

1. Backend (Spring Boot – Java 17):

HospitalManagementSystem Backend/

```
|
|
|— src/
|   |— main/
|       |— java/
|           |— com/
|               |— hospital/
|               |
|               |— HmsApplication.java
|               |
|               |— config/
|                   |— AdminInitializer.java
|                   |— CorsConfig.java
|                   |— JwtFilter.java
|                   |— JwtUtil.java
|                   |— PayPalConfig.java
|                   |— SecurityConfig.java
|                   |— WebConfig.java
|                   |
|                   |— controller/
|                       |— AuthController.java
|                       |— RegistrationController.java
|                       |— ProfileController.java
|                       |— DashboardController.java
|                       |— NotificationController.java
|                       |— AppointmentController.java
|                       |— DoctorController.java
|                       |— PatientController.java
```

```
| | | | | |— DoctorAppointmentController.java
| | | | | |— DoctorDashboardController.java
| | | | | |— PatientDashboardController.java
| | | | | |— PatientRegistrationController.java
| | | | | |— AdminController.java
| | | | | |— AdminDashboardController.java
| | | | | |— AdminManageController.java
| | | | | |— AdminAppointmentController.java
| | | |
| | | | |— dto/
| | | | | |— AuthRequest.java
| | | | | |— AuthResponse.java
| | | | | |— RegisterRequest.java
| | | | | |— LoginRequest.java
| | | | | |— AppointmentRequest.java
| | | | | |— AppointmentResponse.java
| | | | | |— DoctorRegisterRequest.java
| | | | | |— DoctorResponseDTO.java
| | | | | |— PatientRegisterRequest.java
| | | | | |— PatientRequest.java
| | | | | |— PatientDetailsDTO.java
| | | | | |— MonthlyCountDTO.java
| | | | | |— DoctorCaseCountDTO.java
| | | | | |— AdminStatsDTO.java
| | | | | |— AdminUserUpdateRequest.java
| | | |
| | | | |— enums/
| | | | | |— Role.java
| | | | | |— AppointmentStatus.java
| | | | | |— AppointmentTimeSlot.java
| | | | | |— NotificationStatus.java
| | | |
| | | | |— exception/
| | | | | |— GlobalExceptionHandler.java
```

```
| | |
| | |   |— model/
| | |   |   |— User.java
| | |   |   |— Patient.java
| | |   |   |— Doctor.java
| | |   |   |— Appointment.java
| | |   |   |— Notification.java
| | |   |   |— Role.java
| | |
| | |   |— repository/
| | |   |   |— UserRepository.java
| | |   |   |— PatientRepository.java
| | |   |   |— DoctorRepository.java
| | |   |   |— AppointmentRepository.java
| | |   |   |— NotificationRepository.java
| | |
| | |   |— service/
| | |   |   |— AuthService.java
| | |   |   |— UserService.java
| | |   |   |— AdminService.java
| | |   |   |— DashboardService.java
| | |   |   |— AdminDashboardService.java
| | |   |   |— DoctorService.java
| | |   |   |— DoctorDashboardService.java
| | |   |   |— PatientService.java
| | |   |   |— PatientDashboardService.java
| | |   |   |— AppointmentService.java
| | |   |   |— NotificationService.java
| | |   |   |— CustomUserDetailsService.java
| | |   |   |— PayPalService.java
| | |
| | |   |— resources/
| | |   |   |— static/
| | |   |   |— templates/
```



```
| | | └─ application.properties
| |
| └─ test/
| | └─ java/
|
└─ uploads/
|
└─ pom.xml
└─ mvnw
└─ mvnw.cmd
└─ HELP.md
```

2. Frontend (React + JSX):

Hospital Management System Frontend/

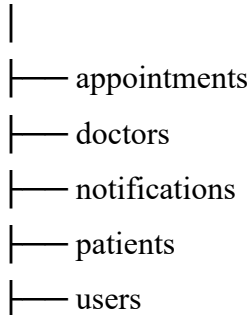
```
|
└─ public/
|   └─ index.html
|
└─ src/
|   |
|   └─ admin/
|       ├── AdminLayout.jsx
|       ├── adminLayout.css
|       ├── Appointments.jsx
|       ├── Appointments.css
|       ├── Manage.jsx
|       ├── Manage.css
|       ├── Sidebar.jsx
|       └─ Sidebar.css
|   |
|   └─ auth/
|       ├── Login.jsx
|       ├── PatientLogin.jsx
|       └─ RegisterDoctor.jsx
```

```
| | | — RegisterPatient.jsx
| | | — RegisterSelect.jsx
| | | — authService.js
| | | — auth.css
| |
| | — components/
| | | — common/
| | | | — Header.jsx
| | | | — Navbar.jsx
| | | | — Sidebar.jsx
| | | | — NotificationBell.jsx
| | | | — Toast.jsx
| | |
| | | — AdminHeader.jsx
| | | — DoctorBarChart.jsx
| | | — MonthlyLineChart.jsx
| | | — CaseStatusPie.jsx
| |
| | — dashboard/
| | | — AdminDashboard.jsx
| | | — AdminDashboard.css
| | | — DoctorDashboard.jsx
| | | — DoctorDashboard.css
| | | — RegisterPage.jsx
| |
| | — doctor/
| | | — DoctorLayout.jsx
| | | — DoctorLayout.css
| | | — DoctorAppointments.jsx
| | | — DoctorAppointments.css
| | | — DoctorProfile.jsx
| | | — DoctorProfile.css
| |
| | — patient/
```

```
| | | — PatientLayout.jsx
| | | — PatientLayout.css
| | | — PatientDashboard.jsx
| | | — PatientDashboard.css
| | | — BookAppointment.jsx
| | | — PatientProfile.jsx
| | | — PatientNotifications.jsx
| | | — PatientHeader.jsx
| |
| | — pages/
| | | — Doctors.jsx
| | | — Doctors.css
| | | — Patients.jsx
| | | — Patients.css
| |
| | — services/
| | | — api.js
| | | — adminService.js
| | | — authService.js
| | | — patientService.js
| |
| | — styles/
| | | — app.css
| | | — common.css
| | | — dashboard.css
| |
| | — App.jsx
| | — main.jsx
| | — index.css
|
| — package.json
| — package-lock.json
| — vite.config.js
| — .gitignore
```

3. Database Structure:

MySQL Database



SOURCE CODE:

BACKEND:

1. Main Application Class:

```
package com.hospital;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication(scanBasePackages = "com.hospital")
public class HmsApplication {

    public static void main(String[] args) {
        SpringApplication.run(HmsApplication.class, args);
    }

}
```

2. SecurityConfig:

```
package com.hospital.config;

import com.hospital.service.CustomUserDetailsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationCo
nfiguration;

import
```

```
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.Arrays;
import java.util.List;
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
    private final JwtFilter jwtFilter;
    private final CustomUserDetailsService userDetailsService;
    public SecurityConfig(JwtFilter jwtFilter,
        CustomUserDetailsService userDetailsService) {
        this.jwtFilter = jwtFilter;
        this.userDetailsService = userDetailsService;
    }
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("http://localhost:5173"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE",
"OPTIONS", "PATCH"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setAllowCredentials(true);
```

```
configuration.setMaxAge(3600L);
UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
source.registerCorsConfiguration("/**", configuration);
return source;
}
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/notifications/**")
                .hasAnyAuthority("ROLE_PATIENT", "ROLE_DOCTOR")
            .requestMatchers("/api/profile/**")
                .hasAnyAuthority("ROLE_PATIENT", "ROLE_DOCTOR")
            .requestMatchers("/api/appointments/**")
                .hasAnyAuthority("ROLE_PATIENT", "ROLE_DOCTOR")
            .requestMatchers("/api/doctor/**")
                .hasAuthority("ROLE_DOCTOR")
            .requestMatchers("/api/patient/**")
                .hasAuthority("ROLE_PATIENT")
            .requestMatchers("/api/admin/**")
                .hasAuthority("ROLE_ADMIN")
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
@Bean
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}  
@Bean  
public AuthenticationManager authenticationManager(  
    AuthenticationConfiguration config) throws Exception {  
    return config.getAuthenticationManager();  
}  
}
```

3. Users.java:

```
package com.hospital.model;  
import com.fasterxml.jackson.annotation.JsonIgnore;  
import com.hospital.enums.Role;  
import jakarta.persistence.*;  
@Entity  
@Table(name = "users")  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    @Column(unique = true, nullable = false)  
    private String email;  
    private String phone;  
    @JsonIgnore  
    private String password;  
    @Enumerated(EnumType.STRING)  
    private Role role;  
  
    private boolean enabled = true;  
  
    public Long getId() { return id; }  
    public String getName() { return name; }
```

```
public String getEmail() { return email; }
public String getPhone() { return phone; }
public String getPassword() { return password; }
public Role getRole() { return role; }
public boolean isEnabled() { return enabled; }

public void setId(Long id) { this.id = id; }
public void setName(String name) { this.name = name; }
public void setEmail(String email) { this.email = email; }
public void setPhone(String phone) { this.phone = phone; }
public void setPassword(String password) { this.password = password; }
public void setRole(Role role) { this.role = role; }
public void setEnabled(boolean enabled) { this.enabled = enabled; }
}
```

4. PaypalService.java:

```
package com.hospital.service;
import java.io.IOException;
import java.util.List;
import org.springframework.stereotype.Service;
import com.paypal.http.HttpResponse;
import com.paypal.http.exceptions.HttpException;
import com.paypal.orders.*;
import com.paypal.core.PayPalHttpClient;
@Service
public class PayPalService {
    private final PayPalHttpClient client;
    public PayPalService(PayPalHttpClient client) {
        this.client = client;
    }
    public String createOrder(Double amount) throws IOException {
        OrderRequest orderRequest = new OrderRequest();
        orderRequest.checkoutPaymentIntent("CAPTURE");
        AmountWithBreakdown amountBreakdown =
```



```
        new AmountWithBreakdown()
        .currencyCode("USD")
        .value(String.format("%.2f", amount));
        PurchaseUnitRequest purchaseUnit =
            new PurchaseUnitRequest().amountWithBreakdown(amountBreakdown);
        orderRequest.purchaseUnits(List.of(purchaseUnit));
        OrdersCreateRequest request =
            new OrdersCreateRequest().requestBody(orderRequest);
        HttpResponseMessage<Order> response = client.execute(request);
        return response.result().id();
    }
    public boolean captureOrder(String orderId) throws IOException {
        OrdersCaptureRequest request =
            new OrdersCaptureRequest(orderId);
        HttpResponseMessage<Order> response = client.execute(request);
        return "COMPLETED".equals(response.result().status());
    }
}
```

FRONTEND:

1. App.JSX:

```
import { Routes, Route } from "react-router-dom";
import PrivateRoute from "../components/PrivateRoute";

/* ===== AUTH ===== */
import Login from "../auth/Login";
import RegisterSelect from "../auth/RegisterSelect";
import RegisterDoctor from "../auth/RegisterDoctor";
import RegisterPatient from "../auth/RegisterPatient";

/* ===== ADMIN ===== */
import AdminLayout from "../admin/AdminLayout";
import AdminDashboard from "../dashboard/AdminDashboard";
```

```
import Doctors from "./pages/Doctors";
import Patients from "./pages/Patients";
import Appointments from "./admin/Appointments";
import Manage from "./admin/Manage";

/* ===== DOCTOR ===== */
import DoctorLayout from "./doctor/DoctorLayout";
import DoctorDashboard from "./dashboard/DoctorDashboard";
import DoctorAppointments from "./doctor/DoctorAppointments";
import DoctorProfile from "./doctor/DoctorProfile";
import DoctorPatientProfile from "./doctor/DoctorPatientProfile";

/* ===== PATIENT ===== */
import PatientLayout from "./patient/PatientLayout";
import PatientDashboard from "./patient/PatientDashboard";
import PatientProfile from "./patient/PatientProfile";
import BookAppointment from "./patient/BookAppointment";
import PatientNotifications from "./patient/PatientNotifications";

function App() {
  return (
    <Routes>

      {/* ===== AUTH ===== */}
      <Route path="/" element={<Login />} />
      <Route path="/login" element={<Login />} />

      <Route path="/register" element={<RegisterSelect />} />
      <Route path="/register/doctor" element={<RegisterDoctor />} />
      <Route path="/register/patient" element={<RegisterPatient />} />

      {/* ===== ADMIN ===== */}
      <Route
        path="/admin"
```

```
    element={
      <PrivateRoute role="ADMIN">
        <AdminLayout />
      </PrivateRoute>
    }
  >
    <Route index element={<AdminDashboard />} />
    <Route path="dashboard" element={<AdminDashboard />} />
    <Route path="doctors" element={<Doctors />} />
    <Route path="patients" element={<Patients />} />
    <Route path="appointments" element={<Appointments />} />

    <Route path="manage" element={<Manage />} />
  </Route>

  { /* ===== DOCTOR ===== */ }
  <Route
    path="/doctor"
    element={
      <PrivateRoute role="DOCTOR">
        <DoctorLayout />
      </PrivateRoute>
    }
  >
    <Route index element={<DoctorDashboard />} />
    <Route path="profile" element={<DoctorProfile />} />
    <Route path="appointments" element={<DoctorAppointments />} />
    <Route path="patient/:patientId" element={<DoctorPatientProfile />} />
  </Route>

  { /* ===== PATIENT ===== */ }
  <Route
    path="/patient"
    element={
```

```
    <PrivateRoute role="PATIENT">
      <PatientLayout />
    </PrivateRoute>
  }
>
  <Route index element={<PatientDashboard />} />
  <Route path="profile" element={<PatientProfile />} />
  <Route path="book" element={<BookAppointment />} />
  <Route path="notifications" element={<PatientNotifications />} />
</Route>

</Routes>
);
}
```

export default App;

2. Index.HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Hospital Management System - Book appointments,
manage health records" />
    <title>Hospital Management System</title>
    <!-- Google Fonts - Inter -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&displa
y=swap" rel="stylesheet">
    <!-- Favicon -->
```

```
<link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-
size='90'> 🏥 </text></svg>" />
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
  <script src="https://www.paypal.com/sdk/js?client-
id=AYAhPWmk0VyIjO3xcoABfhn6XvUtruIKjh6HGCYNVWCKn6Njl1YnNHL4exhFgDPY
el7Lh2LbMT4VhJv3"></script>
</body>
</html>
```

2. API.JS:

```
import axios from "axios";
const api = axios.create({
  baseURL: "http://localhost:8080/api",
  headers: {
    "Content-Type": "application/json",
  },
  withCredentials: false, // JWT in headers, not cookies
})
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);
export default api;
```

CHAPTER - 9

SYSTEM TEST

The primary purpose of software testing is to identify defects and ensure that the developed system meets specified requirements and user expectations. Testing is the process of executing a software system with the intent of discovering errors, validating functionality, and verifying that the application behaves correctly under both normal and abnormal conditions.

In the Hospital Management System, testing plays a crucial role in ensuring that features such as user authentication, patient profile management, appointment scheduling, doctor dashboards, and notifications function reliably. Proper testing ensures system stability, data integrity, security, and overall user satisfaction.

TYPES OF TESTS

9.1 Unit testing:

Unit testing involves testing individual components or modules of the application to ensure that each unit performs its intended functionality correctly. In the **Hospital Management System**, unit testing is carried out after the completion of each module, before integrating it with other modules of the system.

Each unit is tested using predefined input values and expected outputs to validate internal logic, decision-making paths, and error handling mechanisms. Modules such as user authentication, patient management, doctor management, and appointment handling are tested independently. Unit testing helps identify defects at an early stage of development, thereby reducing overall development cost, debugging effort, and improving system reliability.

Examples of Unit Testing in the Project:

- Testing **user registration and login validation** to ensure correct authentication and role-based access.
- Testing **patient profile creation and update** functionalities to verify accurate storage and retrieval of patient details.
- Testing **appointment booking logic** to ensure proper scheduling between patients and doctors.
- Testing **notification generation and status update** functionality for appointment alerts and system messages.

9.2 Integration testing:

Integration testing is conducted to verify the interaction between different modules of the system. After unit testing individual modules, integration testing ensures that the combined modules work together correctly without errors or data inconsistencies.

In the **Hospital Management System**, integration testing focuses on verifying seamless communication between the frontend and backend, interaction between controllers and service layers, and proper data flow between services and the database. This type of testing ensures that integrated components function as a complete system.

Examples

- Interaction between **React frontend** and **Spring Boot REST API**.
- Integration of **appointment scheduling module** with patient and doctor modules
- Integration of **authentication module** with role-based access control (Patient, Doctor, Admin)

9.3 Functional test:

Functional testing is performed to verify that the system functions according to the specified business and technical requirements. It ensures that all features of the Hospital Management System are available and operate as expected.

Functional Testing Focus Areas

- Valid inputs are accepted by the system
- Invalid inputs are properly rejected
- All defined functionalities are executed correctly
- Outputs are generated accurately

Examples

- Verifying login functionality using valid and invalid user credentials
- Ensuring appointment booking displays correct patient, doctor, and time details
- Confirming notifications are generated correctly for appointments and status updates

9.4 System Test

System testing validates the complete and fully integrated **Hospital Management System**. It ensures that the entire system meets both functional and non-functional requirements. This testing is carried out in an environment that closely resembles real-world hospital usage scenarios.

System testing verifies end-to-end workflows such as patient registration, secure login, appointment scheduling, doctor consultation management, and appointment confirmation. It confirms that all modules work together correctly and that the system behaves as expected under normal operating conditions.

9.5 White Box Testing

White box testing is performed with complete knowledge of the internal structure and logic of the software. It focuses on testing internal code components such as control paths, decision conditions, loops, and data flow.

In the **Hospital Management System**, white box testing is used to verify backend logic implemented in Spring Boot, including authentication logic, appointment scheduling rules, role-based access control, and database interaction methods. This testing ensures that the internal code functions correctly and efficiently according to the design specifications.

9.6 Black Box Testing:

Black box testing evaluates the system without considering its internal implementation details. The tester focuses only on providing inputs and verifying outputs based on the system's functional requirements.

Black Box Testing Scenarios

- User login and logout functionality
- Patient profile creation and management
- Appointment booking and status verification
- Notification display for appointments and updates

This type of testing ensures that the **Hospital Management System** behaves correctly from the user's perspective and meets expected functional requirements.

9.7 Integration Testing

Integration testing evaluates the interaction between different modules of the system to ensure they work together correctly. The tester focuses on data flow and communication between integrated components based on system requirements.

Integration Testing Scenarios

- User authentication integrated with role-based dashboard access
- Patient profile linking with user accounts
- Appointment booking integrated with doctor availability
- Notification generation after successful appointment actions his type of testing ensures that the combined components of the **Hospital Management System** function correctly as a complete and integrated system.

9.8 Acceptance Testing

User Acceptance Testing (UAT) is conducted to verify that the **Hospital Management System** meets user expectations and business requirements. End users such as patients, doctors, and administrative staff participate in this testing to validate real-world usage scenarios.

Acceptance testing ensures that the system is user-friendly, fully functional, and ready for deployment. It confirms that all features operate as intended and that the system satisfies hospital operational needs before final implementation.

9.9 Test Strategy and Approach

1 Manual testing is performed to verify the user interface and end-to-end workflows of the Hospital Management System.

2 Functional test cases are written and executed to validate system requirements and expected outputs.

3 Testing is conducted incrementally at each stage of development to identify defects early and ensure system stability.

Test Objectives

- All input fields must work correctly
- Pages must load without delay
- Navigation links must function properly
- Duplicate data entries must be prevented

CHAPTER - 10

RESULTS AND OUTPUTS SCREENS

Backend Configuration and Data Consistency:

During application startup, the Hospital Management System validates the consistency of the database schema with the application's data models. Any pending schema updates or mismatches are identified during initialization through Hibernate and JPA validation mechanisms. These issues are resolved by applying the required schema updates to ensure smooth interaction between the Spring Boot backend services and the MySQL database. This process guarantees data integrity and consistency for patient records, doctor information, appointments, billing details, notifications, and user accounts.

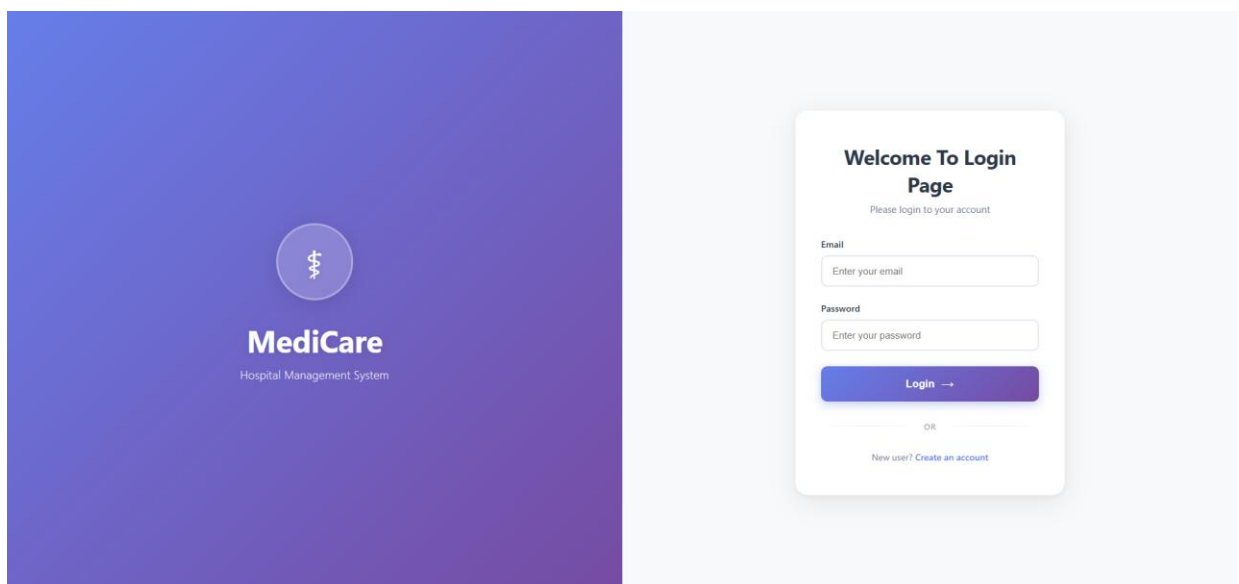


Fig 10.1

- The figure represents the account selection and authentication interface of the Hospital Management System (MediCare).
- It allows users to choose their role (Patient or Doctor) before proceeding with registration or login.
- The role-based access mechanism ensures that users are redirected to their respective dashboards with appropriate permissions.
- Patients can manage appointments and health records, while doctors can manage patient

details, consultations, and schedules.

- This interface strengthens system security by enforcing controlled access and role separation, protecting sensitive medical data and appointment information.

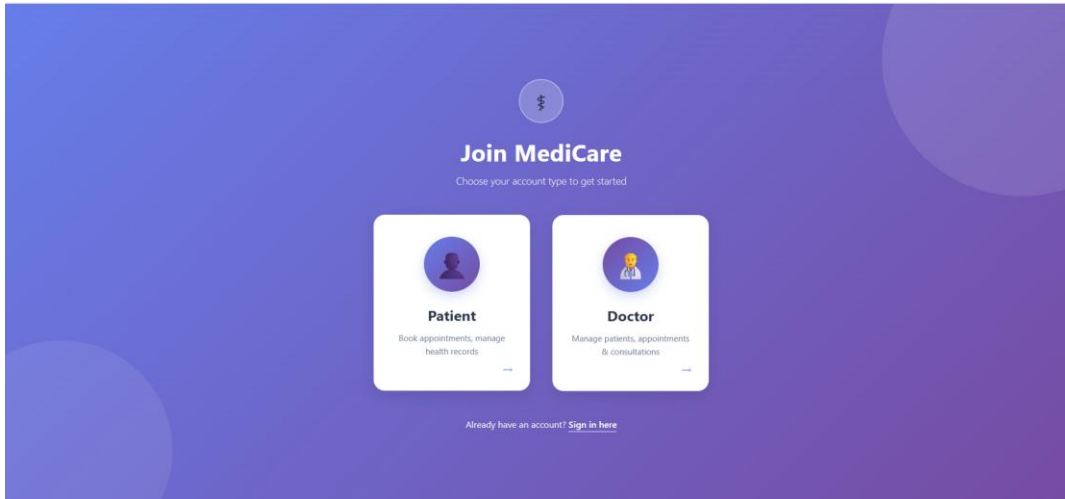


Fig 10.2

- The figure represents the Patient Registration interface of the Hospital Management System.
- The interface allows new patients to create an account by entering essential details such as email, password, name, age, and disease information.
- Role-based registration ensures that patients are securely onboarded and redirected to the Patient Dashboard after successful authentication.
- The controlled registration mechanism prevents unauthorized access and maintains data integrity within the system.
- The consistent color theme and clean layout reflect trust, reliability, and a healthcare-oriented design, enhancing user confidence and usability.

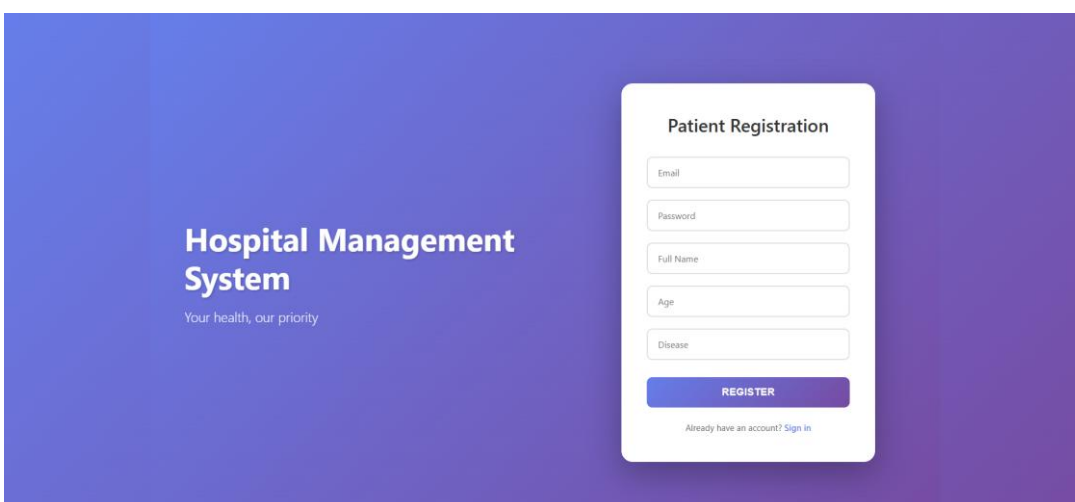


Fig 10.3

- The figure represents the Doctor Registration interface of the Hospital Management System.
- Medical professionals can create their accounts by providing details such as email address, password, full name, and area of specialization.
- This registration process ensures that only verified doctors are onboarded into the system for consultation and appointment management.
- Once registered, doctors gain access to their dedicated Doctor Dashboard, where they can manage patient appointments, view patient details, and update medical records.
- The interface supports seamless interaction between doctors and patients from different locations, enabling efficient online appointment scheduling and consultations.

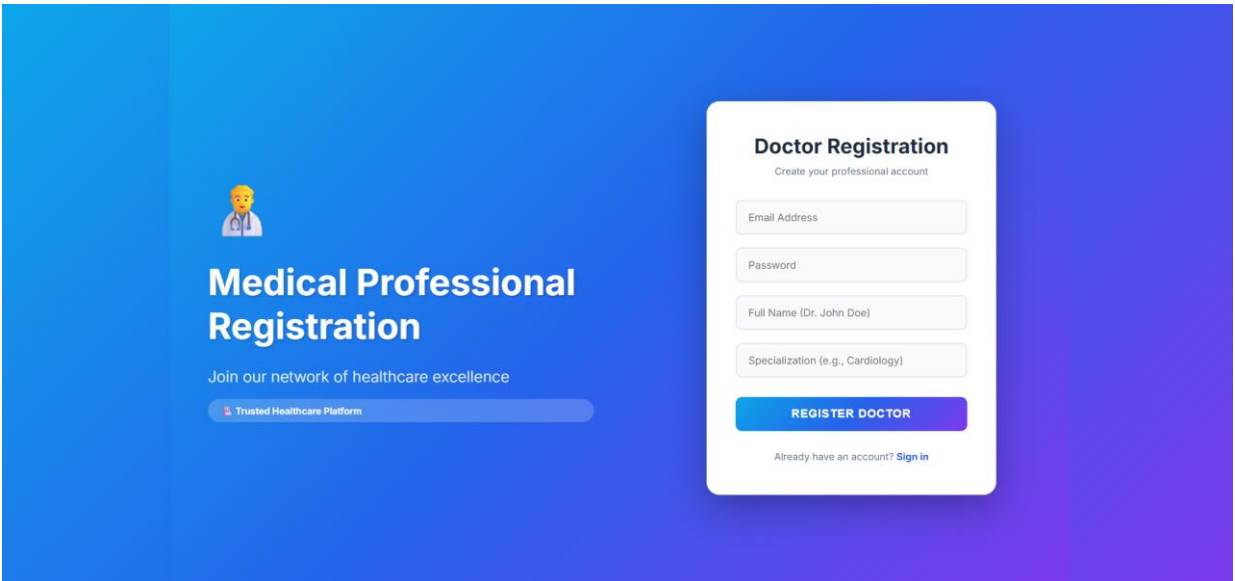


Fig 10.4

Appointment Booking Page (Patient Side)

- This figure represents the Appointment Booking Interface of the Hospital Management System.
- Patients can select doctors, preferred dates, and available time slots for consultations.
- The appointment details are displayed clearly, ensuring easy understanding and tracking.
- The form-based interface simplifies the appointment booking process for users.
- Proper validation and visual feedback confirm successful appointment requests.
- This feature helps patients efficiently schedule consultations while enabling doctors to

manage their availability.

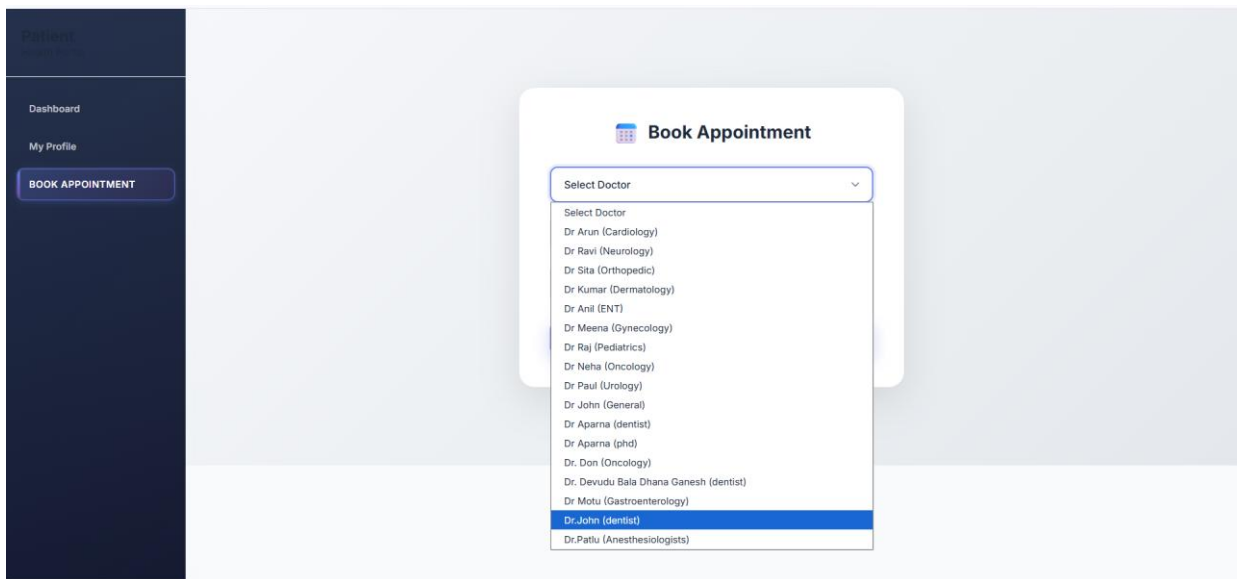


Fig 10.5

Patient Dashboard

- This figure represents the Patient Dashboard interface of the Hospital Management System.
- The dashboard provides a personalized welcome message to the logged-in patient.
- Summary cards display appointment statistics such as Total Appointments, Pending, Approved, and Completed consultations.
- Patients can view a detailed list of their appointments, including Doctor Name, Disease, Appointment Date, and Status.
- Status indicators (Completed, Approved, Pending) help patients easily track appointment progress.
- The Book Appointment button allows patients to schedule new consultations directly from the dashboard.
- Action buttons like View Details enable patients to access complete appointment and medical information.
- The left-side navigation panel provides quick access to features such as Dashboard, Profile, and Book Appointment.
- This dashboard improves usability by centralizing all patient-related activities in a single interface.

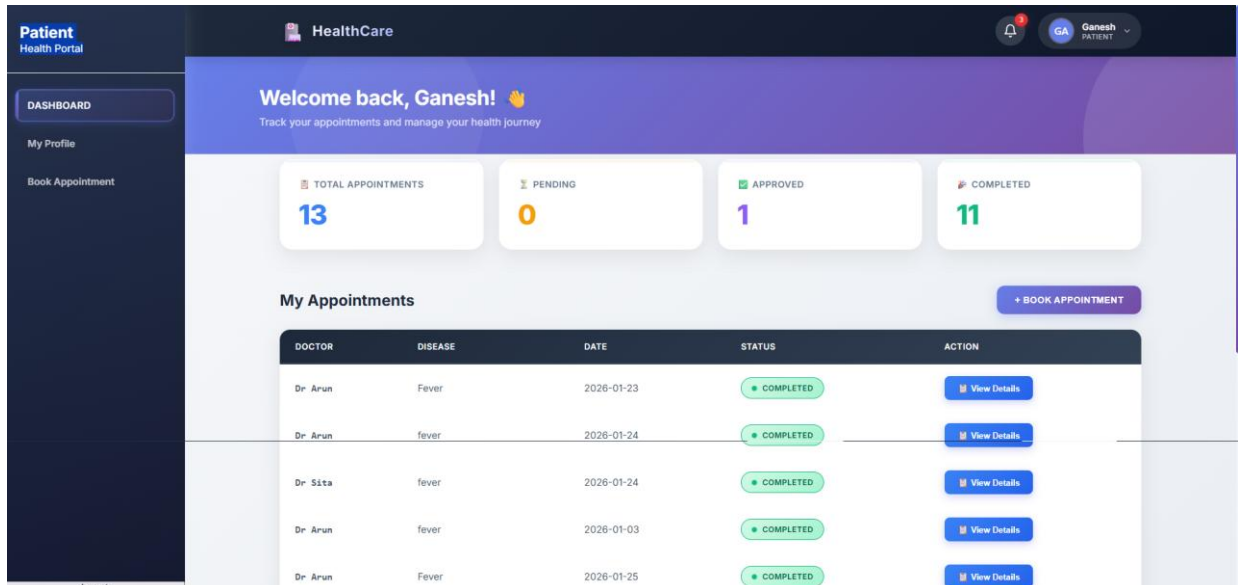


Fig 10.6

Doctor Dashboard (Doctor Portal)

- This figure represents the Doctor Dashboard interface of the Hospital Management System.
- The dashboard provides a centralized overview of the doctor's activities, including appointments, patients, and earnings.
- Key statistics such as Total Appointments, Pending, Approved, Completed Appointments, Total Patients, and Total Earnings are displayed in summary cards for quick reference.
- The interface enables doctors to monitor their daily workload efficiently and track appointment statuses in real time.
- Graphical components like Appointment Status Distribution and Monthly Appointments charts help doctors analyze trends and manage schedules effectively.
- The sidebar navigation allows easy access to features such as Profile Management and Appointment Handling.
- Notification icons alert doctors about new appointments or updates, ensuring timely response.
- Overall, the dashboard is designed to improve productivity, decision-making, and patient care management.

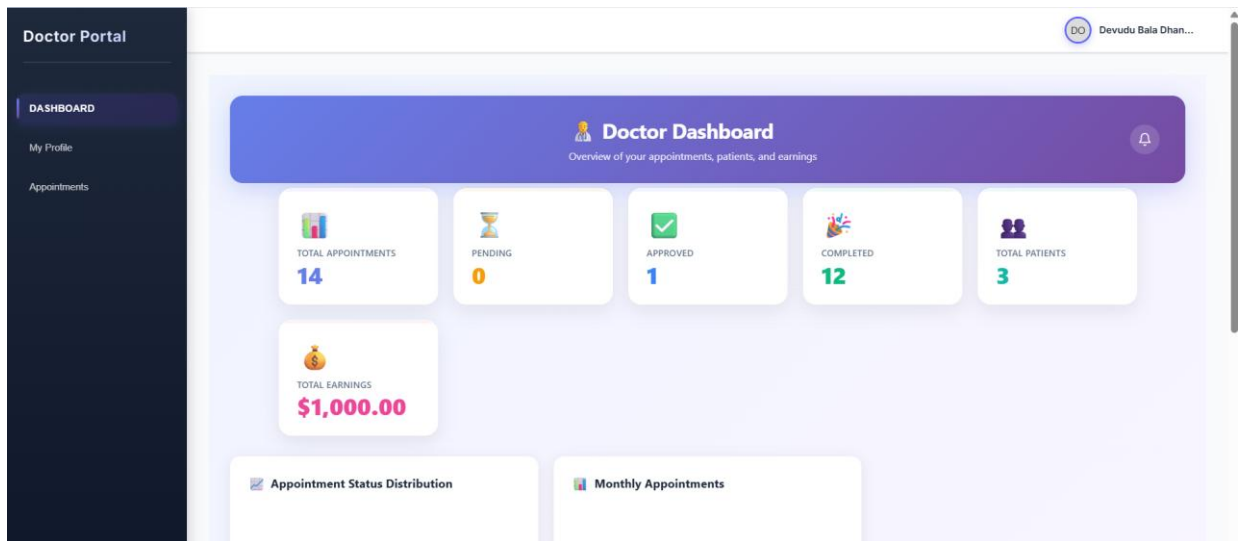


Fig 10.7

Admin Dashboard (HMS Admin Portal)

- This figure represents the Admin Dashboard interface of the Hospital Management System.
- The dashboard provides administrators with a complete overview of hospital operations, including doctors, patients, and appointments.
- Summary cards display key statistics such as Total Doctors, Total Patients, and Total Appointments, enabling quick monitoring of system activity.
- The Case Status chart visually represents the distribution of appointments based on their status (Completed, Pending, and Approved).
- The Doctor-wise Cases bar chart helps administrators analyze workload distribution among doctors.
- The Monthly Appointments graph shows appointment trends over time, supporting data-driven planning and decision-making.
- The sidebar navigation allows administrators to manage Doctors, Patients, Appointments, and System Data efficiently.
- The dashboard enhances administrative control, transparency, and effective hospital management through real-time insights.

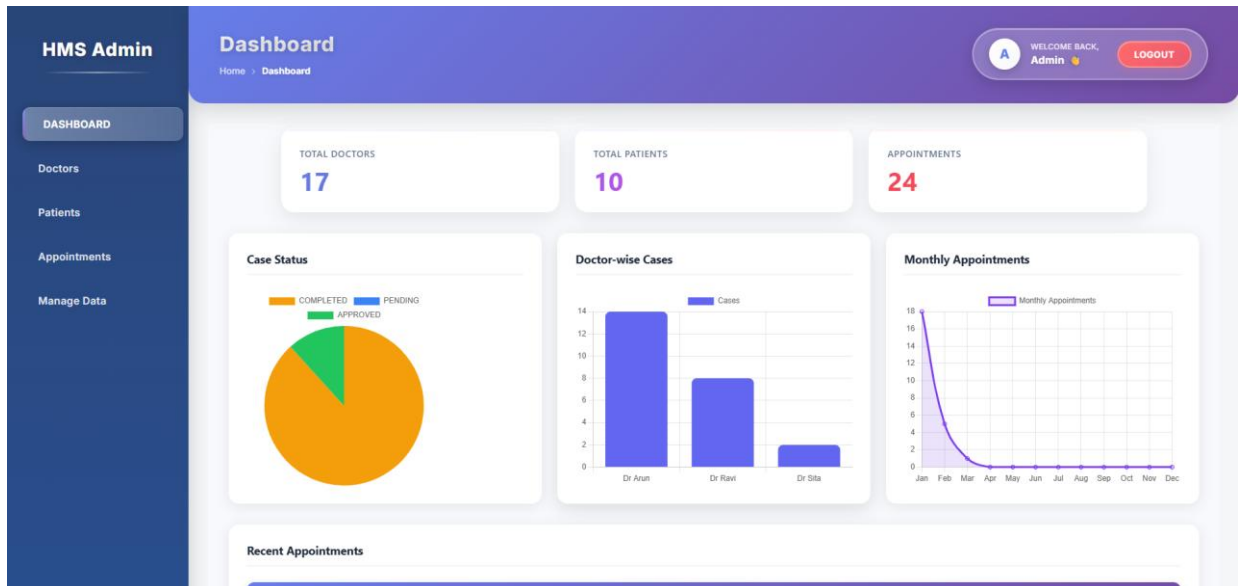


Fig 10..8

Manage Doctors & Patients Page (Admin View)

- This figure represents the Manage Doctors & Patients interface of the Hospital Management System.
- The page allows the administrator to view and manage registered doctors and patients in the system.
- Doctor and patient details such as ID, name, email, phone number, and status are displayed in a tabular format.
- Action buttons like Edit, View, Disable, and Delete enable the admin to efficiently control user accounts.
- Status indicators (Active/Inactive) help monitor availability and system access in real time.
- This interface ensures centralized administration, secure data management, and smooth hospital operations.

The screenshot displays the HMS Admin Dashboard. The left sidebar contains navigation links: Dashboard, Doctors, Patients, Appointments, and a highlighted 'MANAGE DATA' button. The main header shows 'Dashboard' and a user profile for 'Admin' with a 'LOGOUT' button. The main content area is titled 'Manage Doctors & Patients' and features a toggle for 'DOCTORS' (selected) and 'Patients'. Below this is a table listing doctors with columns for ID, NAME, EMAIL, PHONE, STATUS, and ACTIONS. The table contains 7 rows of doctor data, all with a status of 'ACTIVE'. Each row has four action buttons: 'Edit', 'View', 'Disable', and 'Delete'.

ID	NAME	EMAIL	PHONE	STATUS	ACTIONS
2	Devudu Bala Dhana Ganesh	dr1@hospital.com	9573386779	ACTIVE	Edit View Disable Delete
3	Dr Ravi	dr2@hospital.com	9000000003	ACTIVE	Edit View Disable Delete
4	Dr Sita	dr3@hospital.com	9000000004	ACTIVE	Edit View Disable Delete
5	Dr Kumar	dr4@hospital.com	9000000005	ACTIVE	Edit View Disable Delete
6	Dr Anil	dr5@hospital.com	9000000006	ACTIVE	Edit View Disable Delete
7	Dr Meena	dr6@hospital.com	9000000007	ACTIVE	Edit View Disable Delete
8	Dr Raj	dr7@hospital.com	9000000008	ACTIVE	Edit View Disable Delete

Fig 10.9

CHAPTER - 11

CONCLUSION

The Hospital Management System was designed and developed with the objective of providing an efficient, secure, and user-friendly platform for managing hospital-related activities such as patient profiles, medical records, appointment scheduling, and system administration. The system integrates modern web technologies to ensure smooth interaction between patients, doctors, and administrators. By adopting a modular and layered architecture, the application ensures scalability, maintainability, and ease of future enhancements.

The use of **Java and Spring Boot** for backend development provides a robust and secure foundation, while **React with JSX** offers a responsive and dynamic frontend experience. The integration of **MySQL** for persistent data storage and **JWT-based authentication** ensures data integrity and secure role-based access control. Comprehensive testing strategies were employed to validate the functionality, reliability, and performance of the system, confirming that it meets both functional requirements and user expectations.

In the future, the Hospital Management System can be enhanced by incorporating advanced features such as mobile application support, cloud deployment, real-time notifications, and integration with external healthcare services. These improvements would further increase the usability and effectiveness of the system, making it a practical and valuable solution for modern healthcare management.

CHAPTER - 12

REFERENCES

- [1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed., McGraw-Hill Education, 2015.
- [2] I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
- [3] Craig Walls, *Spring Boot in Action*, Manning Publications, 2016.
- [4] Joshua Bloch, *Effective Java*, 3rd ed., Addison-Wesley, 2018.
- [5] Mark Heckler, *Spring Boot: Up and Running*, O'Reilly Media, 2021.
- [6] Facebook Inc., *React – A JavaScript Library for Building User Interfaces*.
Available: <https://reactjs.org>
- [7] Oracle Corporation, *Java Platform, Standard Edition Documentation*.
Available: <https://docs.oracle.com/en/java/>
- [8] MySQL AB, *MySQL Reference Manual*.
Available: <https://dev.mysql.com/doc/>
- [9] Hibernate Team, *Hibernate ORM Documentation*.
- [10] Spring Framework Team, *Spring Security Reference Documentation*.
- [11] Auth0, *JSON Web Token (JWT) Introduction*.
Available: <https://jwt.io/introduction>
- [12] Apache Maven Project, *Maven Documentation*.
Available: <https://maven.apache.org>
- [13] Roy Thomas Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine, 2000.
- [14] Mozilla Developer Network (MDN), *Web Technologies Documentation*.
Available: <https://developer.mozilla.org>
- [15] Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003.

- [16] IEEE, *IEEE Standard for Software Test Documentation*, IEEE Std 829-2008.
- [17] W3C, *HTML and CSS Specifications*.
Available: <https://www.w3.org>
- [18] Postman Inc., *API Testing and Development Documentation*.
Available: <https://www.postman.com>
- [19] IntelliJ IDEA, *JetBrains Documentation*.
Available: <https://www.jetbrains.com>
- [20] Sebastien Deleuze, *Building RESTful Web Services with Spring Boot*, Packt Publishing, 2018.
- [21] Alex Banks and Eve Porcello, *Learning React: Functional Web Development with React and Redux*, 2nd ed., O'Reilly Media, 2020.
- [22] Vlad Mihalcea, *High-Performance Java Persistence*, Vlad Mihalcea Publishing, 2016.
- [23] Gavin King, *Hibernate in Action*, Manning Publications, 2004.
- [24] Google Developers, *Web Application Architecture Best Practices*.
- [25] IEEE Computer Society, *Software Engineering Best Practices*.

Design and Implementation of a Web-Based Hospital Management System Using Modern Full-Stack Technologies

Dr. Meerasharif Sheik¹, Devudu Bala Dhana Ganesh², Adabala Aparna Latha³, Annamneedi Veera Aparna⁴

¹Professor & HOD, Project Supervisor

Department of Computer Science and Engineering
Bonam Venkata Chalamayya Engineering College (BVCEC)
Affiliated to JNTU Kakinada
Odalarevu, Andhra Pradesh, India

^{2,3,4}Department of Computer Science and Engineering
Bonam Venkata Chalamayya Engineering College (BVCEC)
Affiliated to JNTU Kakinada
Odalarevu, Andhra Pradesh, India

Abstract—There has been a growing reliance on digital information systems by health care establishments in order to more effectively manage patient data, schedule appointments, and to manage hospital resources. In addition, conventional (manual or semi-automated) methods of hospitals typically cause data inconsistencies, downtime due to operations not working efficiently, increased administration work, and less accessibility of important information. All of these issues cause a negative effect on both a hospital's level of efficiency and, thus, also provided a lower quality of service to patients. In order to solve these ongoing problems, this paper presents a proposal for the design and development of a web based hospital management system (HMS) to provide a digital platform that will allow the automation of many hospital operations (centralized) and be secure and ease of use. Access is controlled by roles in a manner that supports organized, secure, and safe interaction amongst hospital staff (administrators), physicians (doctors), and patients by establishing the appropriate rules about who can access what information and perform what functions. Hospital administrators are responsible for overseeing day-to-day hospital operations, controlling user access and monitoring appointment scheduling (creating new, editing existing, cancelling and rescheduling). Physicians are responsible for managing their own appointment schedules and caring for patients (consultations). Finally, patients have a means of accessing the hospital's health care services via an interactive web interface that allows them to register to participate, schedule appointments with physicians, and view their appointment history. The web application was created using current, full-stack development technologies: (1) front end - React; (2) back end - Spring Boot (RESTful); and (3) database - MySQL (reliable/persistent data storage). The experimental analysis and functional validation of the digital Health Management System demonstrates that, when implemented appropriately, there are significant increases in operational efficiency, data accuracy, reduction of administrative burden and scalability that are appropriate for small and medium-sized hospitals.

Index Terms—Hospital Management System, Healthcare Information System, Full Stack Development, React, Spring Boot, MySQL, Web Application

I. INTRODUCTION

Hospitals provide crucial health care services and they do this by maintaining and monitoring large amounts of patient and other related information on patients and their medical care. For example, hospitals are generating huge amounts of data every day about their patients, their medical professionals, their forms of treatment, making appointments, billing, etc. Efficient management of all of this data is extremely important to ensure that patients receive accurate diagnoses, timely treatment, and effective administration of their hospital. However, many hospitals continue to maintain their records manually, or they have old programs that don't work well together, which results in duplicate records, inconsistent data, inefficiencies in operations, and more opportunities for mistakes to happen.

Most hospitals have traditionally maintained their record-keeping through paper documents or stored digital files in silos by department. The use of these methods to maintain records makes it difficult for hospital staff to retrieve the appropriate data in a timely manner, increases the time and effort required to maintain the hospital's records and makes it very difficult to keep sensitive records safe from unauthorized access. These issues reduce hospital productivity, increase the time that administrative staff spends working on tasks and lead to lower quality of care for patients. Therefore, there is a rapidly growing demand for a fully integrated and automated solution that will allow hospitals to operate in a safe and efficient manner. A web-based Hospital Management System (HMS) is an online platform that helps in managing hospital operations in a centralized manner, allowing for smooth coordination between various departments and real-time access to important information. With the use of advanced full-stack web technology, HMS systems can be developed to be scalable, secure, and user-friendly, allowing for efficient data management and smooth system operation. The use of client-server architecture

further improves the reliability, maintainability, and efficiency of the system.

This paper presents the design and development of an advanced web-based Hospital Management System that automates various hospital operations through a centralized online system. The main aim of this project is to develop a secure, scalable, and efficient HMS system that makes administrative tasks easier, improves doctor-patient communication, increases data accuracy, and enables better healthcare service delivery. The proposed system is expected to act as a trustworthy online solution for contemporary healthcare organizations looking to enhance operational efficiency and patient satisfaction.

II. PROBLEM STATEMENT

Despite the advancements in healthcare technology, many hospitals are still using manual or semi-automatic systems that are inefficient and difficult to scale. These systems are also facing the following challenges:

- Inefficient appointment scheduling and increased patient waiting times
- Data redundancy and inconsistency across departments
- Lack of centralized and real-time access to patient records
- Increased risk of data loss and security breaches
- High administrative workload and reliance on manual processes
- Limited support for role-based access and authorization

These challenges indicate the need for an integrated and automated Hospital Management System that can efficiently improve operational efficiency, ensure data accuracy, and improve the overall quality of healthcare service delivery.

- Inefficient appointment scheduling
- Data redundancy and inconsistency
- Lack of centralized patient records
- Limited accessibility to real-time data
- Poor role-based security mechanisms

The lack of a comprehensive Hospital Management System may lead to late delivery of healthcare services, poor coordination of activities among different departments, and a high administrative burden. Handling patients' information manually takes a lot of time, and the possibility of errors is high, which may affect the efficiency of hospital staff and patient satisfaction. To address these issues, the project seeks to develop a centralized, role-based web application that can manage hospital activities efficiently by automating major processes. The proposed system will provide administrators, doctors, and patients with secure access to data, allowing for efficient management and fast service delivery in healthcare facilities.

III. OBJECTIVES OF THE SYSTEM

The main aims of the proposed Hospital Management System (HMS) are as follows:

- To automate and simplify the hospital administrative and medical processes using a web-based system
- To ensure secure and role-based access for administrators, doctors, and patients to maintain data privacy and system usage

- To minimize manual work, paperwork, and human errors in managing hospital data and appointments
- To enhance the accuracy, consistency, and availability of patient and hospital data across various departments
- To facilitate effective appointment management and real-time monitoring of hospital operations
- To improve doctor-patient interaction using an intuitive and responsive web interface
- To develop a scalable and modular system architecture that can be expanded to incorporate latest healthcare technologies

IV. RELATED WORK

There have been a few research studies that have analyzed the design and development of Hospital Management Systems with the aim of enhancing hospital management and patient services. The initial hospital management systems were mainly concerned with the digitalization of patient records and the automation of basic operations such as registration and appointment scheduling. Although these systems minimized paperwork and enhanced data storage, they were often not fully integrated and lacked scalability to meet the rising demands of the hospital.

Current research studies stress the need to implement web-based designs and RESTful APIs to enhance system accessibility, compatibility, and efficiency. Web-based HMS systems allow for centralized data management and enable multiple users to access the system concurrently from remote sites. Some research studies also shed light on the advantages of employing contemporary frameworks to improve responsiveness and user experience.

Although technology has made many great improvements in creating an upgraded healthcare IT solution, many current systems have constraints like bad usability, hard to adapt to user needs and behaviours because of how they are designed, unable to be expanded (modular), and bad security mechanisms. Poorly designed access control systems (role based) and not having proper methods for protecting electronic medical data limit opportunities for these existing systems to work properly in the real world of healthcare. The solution that has been proposed will provide a technical answer to these problem areas through full stack technology using good design principles to provide a level of modular architecture. The solution integrates a responsive front end user interface with a secure backend and a well-designed database to produce a solution with greater scalability, longer life cycle/maintenance costs, generate more efficient use, and provide better data security than the currently available solutions in the market.

V. SYSTEM ARCHITECTURE

The proposed system is designed with a three-tier architecture, including the presentation layer, application layer, and data layer. This ensures modularity, scalability, and maintainability of the system. The presentation layer is developed using React. This layer is primarily responsible for handling user interactions, dynamic content display, and user input acquisition

through an interactive interface. This layer communicates with the backend through secure RESTful API calls.

The application layer is developed using Spring Boot. This layer is the central processing unit of the system. It is responsible for request validation, authentication and authorization processes, business logic processing, and interaction between the frontend and the database. This layer provides seamless and secure data transfer between different components of the system through the exposure of RESTful web services.

The data layer is developed using MySQL. This layer is responsible for providing stable and persistent storage for the structured hospital data of users, appointments, and medical records. The three-tier architecture of the proposed system ensures efficient concern handling, enhanced system reliability, and independent scalability of each layer of the system.

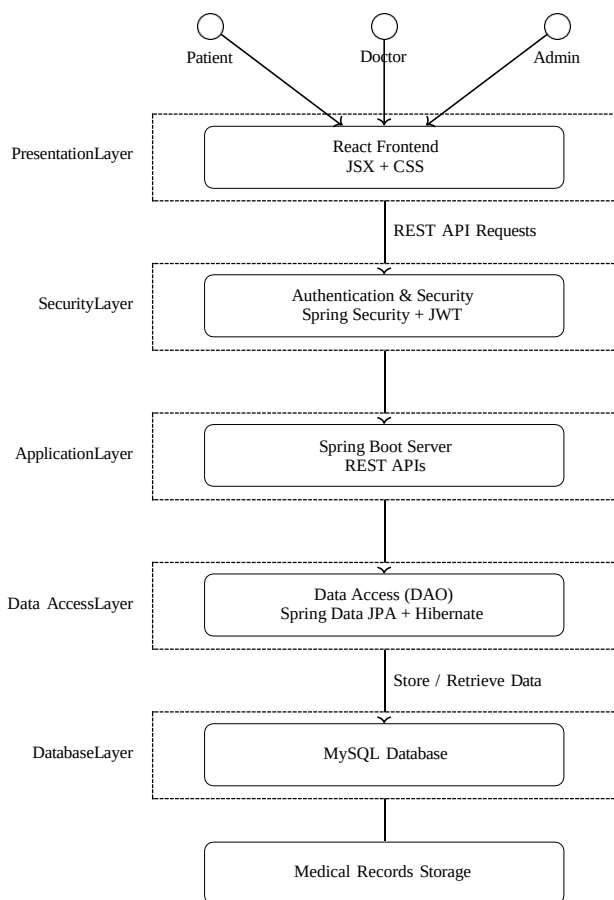


Fig. 1. Architecture Diagram of the Hospital Management System

MySQL is used as the relational database management system for the storage of structured and persistent data. Appropriate database normalization is done to ensure data integrity, avoid data redundancy, and facilitate efficient data retrieval, thus ensuring consistent system performance.

VI. TECHNOLOGY STACK

A. Frontend

React is employed for creating a responsive and dynamic user interface that improves user interaction and overall system usability. The component-based structure of React encourages code reusability, making it efficient for developing modular and scalable frontend components.

B. Backend

Spring Boot is a powerful and scalable backend technology framework for developing RESTful web services. It effectively manages the fundamental server-side operations like authentication, authorization, request handling, and business logic management to ensure a secure and reliable communication flow between the frontend and the database.

C. Database

MySQL is used as the relational database management system for the storage of structured and persistent data. Appropriate database normalization is done to ensure data integrity, avoid data redundancy, and facilitate efficient data retrieval, thus ensuring consistent system performance.

VII. MODULE DESCRIPTION

A. Admin Module

The Admin module serves as the central controller of the system, offering administrators full control over hospital data and system users. Administrators are able to monitor doctor and patient information, manage appointments, and control system operations to ensure a smooth flow of activities within the hospital management system.

Responsibilities:

- Manage doctors and patients
- Monitor appointments
- View system analytics
- Control user access

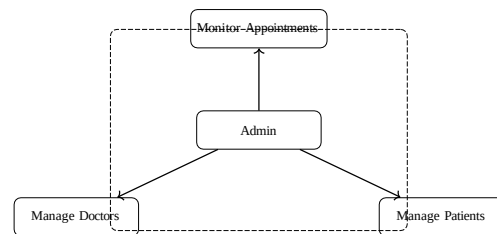


Fig. 2. Admin Module Functional Diagram

B. Doctor Module

The doctors interact with the patients and handle their appointments online using an interactive user interface. The system allows the doctors to view appointments, change the status of the consultations, and access patient details in real time. The proposed system reduces the workload of the doctors by minimizing the documentation process, and thus the doctors

have more time to concentrate on the patients and provide better healthcare services. **Responsibilities:**

- View appointment schedules
- Update consultation status
- Access patient and doctor details

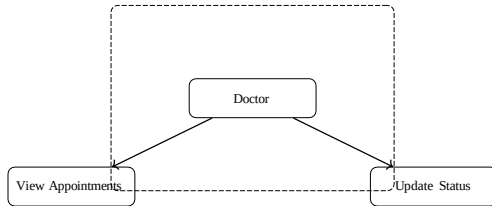


Fig. 3. Doctor Module Functional Diagram

C. Patient Module

The patients are able to access the services of the hospital through a web portal that is secure and allows the patients to register, log in, book appointments with doctors, and view their appointment history. The web portal provides privacy and secure communication for the patients. **Responsibilities:**

- Register and login
- Book appointments and get confirmation notification
- Pay amount and get appointment
- View prescription
- View appointment history

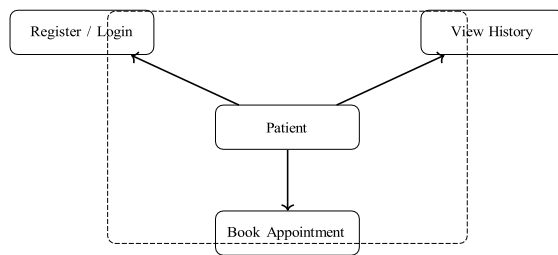


Fig. 4. Patient Module Functional Diagram

VIII. SYSTEM INTERFACE AND SCREENSHOTS

This section provides some screenshots of the developed Hospital Management System to demonstrate the usability of the system. The screenshots show how the administrators, doctors, and patients use the system to accomplish their respective tasks. The screenshots emphasize the important features of the system, which include authentication, appointment system, dashboard analysis, and patient interaction components, thereby verifying the successful implementation of the proposed system.

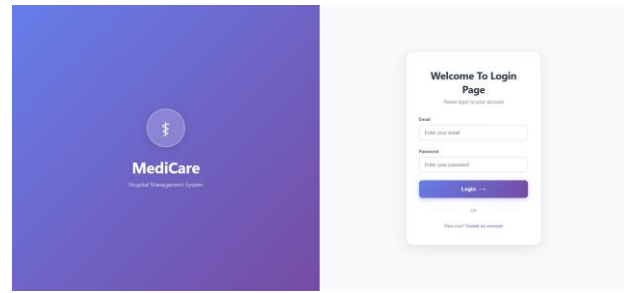


Fig. 5. Login Interface

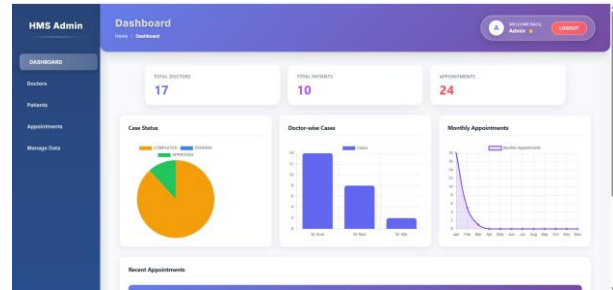


Fig. 6. Admin Dashboard

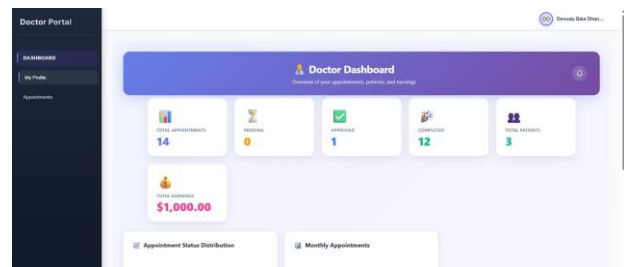


Fig. 7. Doctor Dashboard

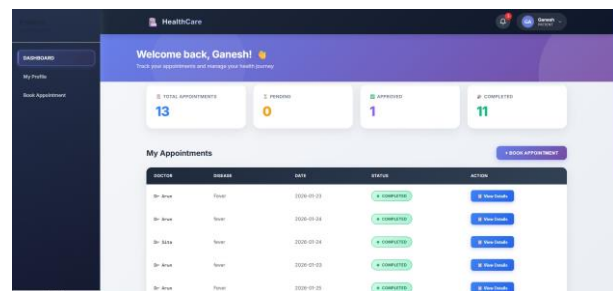


Fig. 8. Patient Dashboard

IX. RESULTS AND DISCUSSION

The proposed Hospital Management System was tested in a real-world environment involving administrators, doctors, and patients. The functional testing proved that the basic functionality of patient registration, appointment scheduling, doctor management, and administrative monitoring was performed efficiently and with no data integrity issues. The role-

based access control system enabled secure and authorized interactions for all roles.

The system's dashboard offered real-time information about appointment status, user activity, and hospital operations, which helped in better monitoring and decision-making. The administrators could efficiently manage doctors, patients, and appointments, and doctors could monitor their appointments and update the status of appointments seamlessly. Patients had a friendly interface that made appointment scheduling and viewing history easy. The system performed well and proved its appropriateness for implementation in a healthcare setting.

X. FUTURE ENHANCEMENTS

Future improvements to the proposed Hospital Management System can be made in the areas of improving accessibility, intelligence, and scalability. A mobile app for Android and iOS devices can be developed to enable patients and doctors to access hospital services at any time and from any location, thus improving the usability of the system. Cloud hosting of the system using platforms like AWS or Azure can further improve the scalability, availability, and backup features of the system, making it appropriate for large-scale healthcare setups.

The inclusion of artificial intelligence algorithms can greatly improve the efficiency of the system by allowing intelligent appointment management, workload prediction, and patient flow management. AI-driven analytics can help hospital administrators make informed decisions by analyzing usage patterns and resource utilization trends. Moreover, integration with national digital health platforms, such as government-supported health record platforms, can help the system facilitate secure data sharing and healthcare standard compliance. These improvements will help the system evolve into a smart and future-proof healthcare management system that can support advanced digital healthcare services.

XI. CONCLUSION

This paper has discussed a comprehensive and scalable web-based Hospital Management System developed using modern full-stack technologies. The proposed system efficiently automates important hospital operations like patient registration, appointment scheduling, doctor management, and administrative monitoring using a centralized digital platform. By implementing a role-based access control system, the proposed system provides secure and organized interaction between administrators, doctors, and patients, thus securing confidential medical information and ensuring data integrity.

The combination of a responsive React-based front-end with a powerful Spring Boot-based back-end provides efficient communication using RESTful services, and the implementation of a MySQL database provides reliable and efficient data storage. The proposed system architecture is highly maintainable, extensible, and scalable, thus making the application ready for real-world deployment in small to medium-scale healthcare organizations. Practical testing and experimental analysis show that the proposed system is highly effective

in reducing manual work, eliminating operational delays, and improving overall user experience.

In conclusion, the proposed Hospital Management System is a reliable, secure, and user-friendly digital platform that enhances the efficiency of hospital administration and promotes efficient healthcare service delivery. The proposed system showcases the potential of modern web technologies in efficiently transforming traditional healthcare management into a smart digital environment.

REFERENCES

- [1] K. Nishanthan, S. Munde, S. Maskar, and S. Kedar, "The Hospital Management System," *International Journal of Engineering and Management Research (IJEMR)*, vol. 12, no. 3, pp. 45–50, 2022.
- [2] S. Dongre and N. Thadani, "Survey on Hospital Management Systems," *International Journal of Advance Research in Computer Science and Engineering (IJARISE)*, vol. 7, no. 2, pp. 112–118, 2021.
- [3] Government of India, "Ayushman Bharat Digital Mission," Ministry of Health and Family Welfare, 2023. [Online]. Available: <https://abdm.gov.in>
- [4] J. Smith and A. Johnson, "Design and Implementation of a Web-Based Hospital Management System," *Journal of Healthcare Informatics*, vol. 8, no. 1, pp. 25–32, 2020.
- [5] A. Kumar, R. Sharma, and P. Verma, "A Review on Hospital Management Systems," *International Journal of Computer Applications*, vol. 179, no. 12, pp. 15–20, 2018.
- [6] R. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2019.
- [7] S. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson Education, 2016.
- [8] M. Chen, S. Mao, and Y. Liu, "Big Data Analytics in Healthcare," *IEEE Network*, vol. 31, no. 1, pp. 18–25, 2017.
- [9] P. Gupta and D. Patel, "Secure Web-Based Healthcare Information Systems," *International Journal of Information Security*, vol. 14, no. 4, pp. 345–352, 2019.
- [10] World Health Organization, "Digital Health," WHO, 2022. [Online]. Available: <https://www.who.int/health-topics/digital-health>
- [11] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2003.
- [12] R. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
- [13] Oracle Corporation, "MySQL 8.0 Reference Manual," 2023. [Online]. Available: <https://dev.mysql.com/doc/>
- [14] Spring Team, "Spring Boot Reference Documentation," 2023. [Online]. Available: <https://spring.io/projects/spring-boot>
- [15] Meta Platforms Inc., "React Documentation," 2023. [Online]. Available: <https://react.dev>

BIOGRAPHIES OF AUTHORS



Dr. Meerasharif Sheik is a distinguished academician and seasoned administrator, currently serving as Professor and Head of the Department for Computer Science and Engineering Information Technology at Bonam Venkata Chalamayya Engineering College (Autonomous), Odalarevu, Amalapuram. With an illustrious academic career spanning over two decades, he has made remarkable contributions to teaching, research, and institutional development. Dr. Meerasharif Sheik brings with him 20+ years of rich teaching experience and 7 years of dedicated research experience as a Research Scholar. His diverse and cutting-edge research interests encompass Quantum Computing, Computer Networks, Mobile Ad hoc Networks (MANETs), Vehicular Ad hoc Networks (VANETs), Cryptography and Network Security, Data Mining, Machine Learning, Deep Learning, Artificial Intelligence, Neural Networks, Data Science, Image Processing, Advanced Data Structures, Algorithm Analysis, and Ethical Hacking. A prolific researcher, he has published 13 research papers in reputed international journals indexed in Scopus and in prestigious national and international conference proceedings. In addition, he has filed three patents, reflecting his innovative mindset and contribution to technological advancement. He is also the author of a book on Neural Networks and Deep Learning, adding significant value to academic literature. Dr. Meerasharif Sheik has actively participated in and presented research papers at numerous national and international conferences, and has attended as well as conducted several Workshops, Faculty Development Programs (FDPs), Seminars, and Refresher Courses, thereby continuously updating his academic and professional competencies. An inspiring mentor, he has successfully guided 38 B.Tech projects and 14 M.Tech projects, nurturing young minds and shaping future professionals and researchers. Beyond academics and research, he possesses extensive administrative and leadership experience, having served in key roles such as Project Coordinator, Chairman of Board of Studies (BoS), Controller of Examinations, and Head of the Department for multiple programs including CSE, IT, AI ML, CSM, AI DS, and MCA in reputed institutions. His dynamic leadership, academic vision, and commitment to excellence continue to contribute significantly to institutional growth and academic innovation.

Email: sharief.bvce@bvcgroup.in



Devudu Bala Dhana Ganesh is currently pursuing a Bachelor of Technology (B.Tech) degree in Computer Science and Engineering at Bonam Venkata Chalamayya Engineering College (BVCEC), Amalapuram, Andhra Pradesh, India. His primary areas of interest include full-stack web development, software engineering, and healthcare information systems. He has acquired strong practical experience in designing and developing web-based applications using modern technologies such as React, Spring Boot, and MySQL.

Email: devuduganesh58@gmail.com

ORCID: 0009-0004-8936-4288



Adabala Aparna Latha is currently pursuing her Bachelor of Technology (B.Tech) degree in Computer Science and Engineering at Bonam Venkata Chalamayya Engineering College (BVCEC), Amalapuram, Andhra Pradesh, India. Her academic interests include web technologies, database management systems, and application development. She has gained practical exposure to designing and developing user-centric web interfaces and working with relational databases. She aims to apply her knowledge to develop efficient and scalable software applications.

Email: 22221a0501@gmail.com

ORCID: 0009-0008-2847-161X



Annamneedi Veera Aparna is currently pursuing her Bachelor of Technology (B.Tech) degree in Computer Science and Engineering at Bonam Venkata Chalamayya Engineering College (BVCEC), Amalapuram, Andhra Pradesh, India. Her areas of interest include full-stack development, data management, and information systems. She has actively contributed to system analysis, testing, and validation of web-based applications, ensuring functional correctness and usability. She is keen on exploring modern software development practices.

Email: aparnaannamneedi07@gmail.com

ORCID: 0009-0006-8635-7941

Copyright & License:

© Authors retain the copyright of this article. This work is published under the Creative Commons Attribution 4.0 International License (CC BY 4.0), permitting unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.



International Journal of Versatile Research and Analysis

IJVRA.ORG | ISSN: 2984-8903

An International Open Access, Peer - Reviewed, Refereed Journal

The Board of

International Journal of Versatile Research and Analysis

Is hereby awarding this certificate to

Dr. Meerasharif Sheik

In recognition of the publication of the paper entitled

**Design and Implementation of a Web-Based Hospital Management System
Using Modern Full-Stack Technologies**

Published in Volume 4 Issue 2, February-2026, 20-02-2026, | Impact Factor: 9.12 by Google Scholar

Co-Authors - Devudu Bala Dhana Ganesh, Adabala Aparna Latha, Annamneedi Veera Aparna

Paper ID - IJVRA2602295



Registration ID - 700854

Transparent Peer Review Based on UGC CARE Approved Journal Publication and Global Indexing

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

IJVRA - International Journal of Versatile Research and Analysis

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.ijvra.org | Email: editor@ijvra.org | ESTD: 2023

Editor-In Chief

Certificate of Publication

IJVRA | ISSN : 2984-8903



International Journal of Versatile Research and Analysis

IJVRA.ORG | ISSN: 2984-8903

An International Open Access, Peer - Reviewed, Refereed Journal

The Board of

International Journal of Versatile Research and Analysis

Is hereby awarding this certificate to

Devudu Bala Dhana Ganesh

In recognition of the publication of the paper entitled

**Design and Implementation of a Web-Based Hospital Management System
Using Modern Full-Stack Technologies**

Published in Volume 4 Issue 2, February-2026, 20-02-2026, | Impact Factor: 9.12 by Google Scholar

Co-Authors - Dr. Meerasharif Sheik, Adabala Aparna Latha, Annamneedi Veera Aparna

Paper ID - IJVRA2602295



Registration ID - 700854

Transparent Peer Review Based on UGC CARE Approved Journal Publication and Global Indexing

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

IJVRA - International Journal of Versatile Research and Analysis

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.ijvra.org | Email: editor@ijvra.org | ESTD: 2023

Editor-In Chief

Certificate of Publication

IJVRA | ISSN : 2984-8903



International Journal of Versatile Research and Analysis

IJVRA.ORG | ISSN: 2984-8903

An International Open Access, Peer - Reviewed, Refereed Journal

The Board of

International Journal of Versatile Research and Analysis

Is hereby awarding this certificate to

Adabala Aparna Latha

In recognition of the publication of the paper entitled

**Design and Implementation of a Web-Based Hospital Management System
Using Modern Full-Stack Technologies**

Published in Volume 4 Issue 2, February-2026, 20-02-2026, | Impact Factor: 9.12 by Google Scholar

Co-Authors - Dr. Meerasharif Sheik, Devudu Bala Dhana Ganesh, Annamneedi Veera Aparna

Paper ID - IJVRA2602295



Registration ID - 700854

Editor-In Chief

Transparent Peer Review Based on UGC CARE Approved Journal Publication and Global Indexing

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

IJVRA - International Journal of Versatile Research and Analysis

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.ijvra.org | Email: editor@ijvra.org | ESTD: 2023

Certificate of Publication

IJVRA | ISSN : 2984-8903



International Journal of Versatile Research and Analysis

IJVRA.ORG | ISSN: 2984-8903

An International Open Access, Peer - Reviewed, Refereed Journal

The Board of

International Journal of Versatile Research and Analysis

Is hereby awarding this certificate to

Annamneedi Veera Aparna

In recognition of the publication of the paper entitled

**Design and Implementation of a Web-Based Hospital Management System
Using Modern Full-Stack Technologies**

Published in Volume 4 Issue 2, February-2026, 20-02-2026, | Impact Factor: 9.12 by Google Scholar

Co-Authors - Dr. Meerasharif Sheik, Devudu Bala Dhana Ganesh, Adabala Aparna Latha

Paper ID - IJVRA2602295



Registration ID - 700854

Transparent Peer Review Based on UGC CARE Approved Journal Publication and Global Indexing

An International Scholarly, Open Access, Multi-disciplinary, Monthly, Indexing in all Major Database & Metadata, Citation Generator

IJVRA - International Journal of Versatile Research and Analysis

An International Scholarly, Open Access, Multi-disciplinary, Indexed Journal

Website: www.ijvra.org | Email: editor@ijvra.org | ESTD: 2023

Editor-In Chief

Certificate of Publication

IJVRA | ISSN : 2984-8903